Strategy synthesis of LIA-definable impartial combinatorial games

Liangda Fang[a,d], Jiahao Luo[a], Meihong Yang[a], Sijie Yin[a], Shiguang Feng[b], Liping Xiong[c], Quanlong Guan[a]

[a] *Jinan University, Guangzhou, 510632, Guangdong, China*
[b] *Sun Yat-sen University, Guangzhou, 510006, Guangdong, China*
[c] *Wuyi University, Jiangmen, 529020, Guangdong, China*
[d] *Pazhou Lab, Guangzhou, 510335, Guangdong, China*

Abstract

Strategy representation and reasoning has recently received much attention in artificial intelligence. Impartial combinatorial games (ICGs) are a type of elementary and fundamental games in game theory. One of the challenging problems of ICGs is to construct winning strategies, particularly, generalized winning strategies for possibly infinitely many instances of ICGs. In this paper, we investigate synthesizing generalized winning strategies for ICGs. To this end, we first propose a logical framework to formalize ICGs based on the linear integer arithmetic fragment of numeric part of PDDL. We then propose two approach to generating the candidate formula that exactly captures the states in which the player can force to win: one adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Furthermore, we compute winning strategies for ICGs based on the winning formula. Experimental results on several games demonstrate the effectiveness of our approach.

Keywords: Impartial Combinatorial Games, Strategy Synthesis, Partial MaxSAT, Linear Integer Arithmetic

1. Introduction

Strategy representation and reasoning has recently received much attention in artificial intelligence, particularly, multi-agent systems and game theory

[1, 2, 3]. In the area of game theory, one class of the elementary and fundamental games is impartial combinatorial games (ICGs) that studies sequential games with perfect information [4]. Various ICGs (*e.g.*, Nim [5], Wythoff [6], Chomp [7] and Empty-and-Divide [8]), and their variants and extensions [9, 10] have been proposed and analyzed. The following is Take-away-3 game [11].

Example 1. Initially, a pile of chips is on a table. In a move, a player can take at most 3 chips from the table and must take at least 1 chip. The loser is the player who has no chip can be taken. □

In general, an ICG contains infinite number of states. Infinite-state ICGs pose two challenging problems. The first problem is to identify the exact characteristic of the set of winning states, that are the states in which the player can force to win. This problem has always been a long-standing topic of ICGs [12, 9, 13, 14]. For example, a player can force to win in any state where the number v of remaining chip is not divisible by 4 (more precisely, $v \not\equiv_4 0$) in Take-away-3 game. It is impossible that a player have a winning strategy in other states where $v \not\equiv_4 0$.

The second problem is to synthesize generalized winning strategies for infinite many winning states. An ICG instance is an ICG together with a specific initial state. A winning strategy for an ICG instance can be computed via backward search algorithm [15]. However, this strategy holds for only some instances but not (infinitely) many instances. This is illustrated in the Take-away-3 game as follows. Assume that the number of chips is initially 7. In the first round, the first player takes 3 chips. If the other player takes 1 (resp. 2 and 3) chips, then she chooses to take 3 (resp. 2 and 1) chips in the next round. Following the above strategy, the first player finally wins no matter how the other player does. But for another instance where the initial state is $v = 5$, after the first player takes 3 chips, the number of the remaining chips is 2. Then, the other player takes 2 chips and no chips is on the table. Finally, the loser is the first player. However, the first player can force to win in the initial state $v = 5$ since $5 \not\equiv_4 0$. Hence, the above strategy does not work for some instances.

In this paper, we propose an automatic approach to solving the above two problems of infinite-state ICGs. The contributions of this paper contain:

1. We propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [16].
2. We define an arithmetic formula, called the winning formula, that is a symbolic representation that exactly captures the states in which the player forces to win. We propose a counterexample-guided approach to synthesizing the winning formula. It first attempts to generate a candidate formula ϕ via a finite set of states, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the winning formula ϕ. Otherwise, a set of new states that are witnesses of its failure will be provided, and used to create a new candidate formula. The above steps repeat until the winning formula is discovered.
3. One crucial task of the counterexample-guided approach is to generate a candidate formula that is consistent with two generated sets of winning states and losing states. We design two methods to accomplish this task. The first method we adopt is a simple enumerative approach proposed in [17]. The main idea is to generate candidate formulas by induction on the size until it finds a desired one. However this approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula is relatively large. To accelerate the generation of candidate formulas, we propose the second method, and formulate it as an exact binary classification problem with all atoms as classification attributes and encode each classification problem as an instance of the partial MaxSAT problem. We can efficiently acquire the winning formula of large size with the aid of off-the-shelf partial MaxSAT solvers.
4. The another core task of the counterexample-guided approach is to check if the candidate formula is the winning formula. We design several con-

straints for the winning formula that is represented by linear integer arithmetic. The candidate formula is correct iff all of the constraints are valid. Therefore, this task can be accomplished by the state-of-the-art SMT solvers: Z3 [18], CVC4 [19] and MathSMT 5 [20].

5. To synthesize the winning strategy, we develop a counterexample-guided approach from a generated winning formula. It first produces a finite set of actions that transforms at least one winning state to a losing state, and then determines the condition formula that captures the set of winning states from which performing the action leads to a losing state for each action. A rule is a pair of an action and its corresponding condition formula, and a strategy consists of a set of rules. To check the correctness of a candidate strategy, we design the constraints for each of its rule.

6. We establish a standard benchmark of ICGs composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. Each test case is formalized in a PDDL file. We also evaluate our proposed approach on this benchmark, demonstrating that the effectiveness of our proposed approach.

This paper is an extended and revised version of [21] and [22]. We provide the complete proofs of all propositions, lemmas, and theorems in [22] together with Theorems **??** and **??**. Theorem **??** establishes the undecidability of determining if an LIA-definable game is an LIA-definable ICG, whereas Theorem **??** shows the decidability of determining if a state is winning in an LIA-definable ICG. In [21], the strategy synthesis approach first splits it into a set of arithmetic terms such that their disjunction is equivalent to the winning formula according to a set of simple division rules. Then, for each term, it uses the enumerative algorithm to obtain the desired action such that the player reaches a losing state from any state satisfying the term via performing this action. However, it is not guaranteed that each term has a proper action, and the above approach fails to compute a winning strategy. In addition, the experimental results show that the strategy synthesis approach proposed in [22] combined with the partial

MaxSAT-based method to generate candidate formulas outperforms the one in [21]. Hence, this paper does not include the illustration and experimental comparison of the strategy synthesis approach proposed in [21].

(Structure ??) The rest of the paper is organized as follows. In Section 2, we provide some necessary preliminaries, including the concepts of linear integer arithmetic (LIA), the maximum satisfiability problem, and ICGs. In Section 3, we propose a logical framework to formalize ICGs via the linear integer arithmetic fragment of Planning Domain Definition Language (PDDL) [16]. In Section 4, we introduce the exact binary classification problem and propose two approach that reduce this problem to the enumerative approach proposed in [17] and partial MaxSAT problems, respectively. In Section 5, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas and strategies for impartial combinatorial games. In Section 6, we evaluate our proposed approaches on the publicly available benchmark composed of 5 categories of ICGs: Subtraction, Nim, Wythoff, Welter, and Chomp for a total of $3,720$ test cases. (??) Finally, we offer related works in Section 7, and close with conclusions and future works in Section 8.

2. Preliminaries

In this section, we introduce the syntax of linear integer arithmetic (LIA), the partial maximum satisfiability (MaxSAT) problem, and impartial combinatorial games (ICGs).

2.1. Linear integer arithmetic

Let $\mathcal{N}$ be the set of integers and $\mathcal{V}$ the set of variables. *Linear integer arithmetic expressions* (`Exp`), *atoms* (`Atom`) and *formulas* (`Form`) are defined by the following grammar:

$$e, e_1, e_2 \in \texttt{Exp} :: c \mid v \mid e_1 + e_2 \mid e_1 - e_2$$

$$l \in \texttt{Atom} :: e_1 = e_2 \mid e_1 < e_2 \mid e_1 \leq e_2 \mid e \equiv_{c_1} c_2$$

$$\phi, \phi_1, \phi_2 \in \texttt{Form} :: \top \mid \bot \mid l \mid \neg\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \forall v\phi \mid \exists v\phi$$

where $c, c_1, c_2 \in \mathcal{N}$ and $v \in \mathcal{V}$.

The atom $e \equiv_{c_1} c_2$ means that e and c_2 are congruence modulo c_1, that is, $e - c_2$ is divisible by c_1. We use $e_1 \neq e_2$ for $\neg(e_1 = e_2)$, $e_1 > e_2$ for $\neg(e_1 \leq e_2)$, $e_1 \geq e_2$ for $\neg(e_1 < e_2)$, $e \not\equiv_{c_1} c_2$ for $\neg(e \equiv_{c_1} c_2)$, and $\phi_1 \rightarrow \phi_2$ for $\neg\phi_1 \vee \phi_2$. We use $|e|$ (resp. $|\phi|$) to represent the size of an LIA expression e (resp. LIA formula ϕ), that is, the number of occurrences of integers, variables and connectives in e (resp. ϕ). For example, the formula $x = 3$ has size 3. Given a set Ψ of atoms, we say a formula ϕ is based on Ψ, iff it is built from the set Ψ, the connectives $\neg, \vee, \wedge, \rightarrow$ and two Boolean constants $\top$ and $\bot$. For example, suppose that $\Psi = \{x = 1, x > 2\}$. The formula $\neg(x = 1 \vee x > 2)$ is based on Ψ, but $x \leq 3 \wedge x > 2$ is not as the atom $x \leq 3 \notin \Psi$.

It is well-known that LIA allows for quantifier elimination, that is, any LIA formula can be transformed into an equivalent quantifier-free formula [23, 24]. Let $\mathcal{U}$ be a subset $\{u_1, \cdots, u_n\}$ of variables. We use $\texttt{Exp}_{\mathcal{U}}$ for the set of expressions over $\mathcal{U}$. We use $\texttt{Form}_{\mathcal{U}}$ for the set of formulas of which free variables are $\mathcal{U}$. We use $\forall\mathcal{U}\phi$ for $\forall u_1 \cdots \forall u_n \phi$ and $\exists\mathcal{U}\phi$ for $\exists u_1 \cdots \exists u_n \phi$.

We say a set S of n-tuples of integers is *LIA-definable*, iff there is an LIA formula $\phi(x_1, \cdots, x_n)$ s.t. for every n-tuple $(z_1, \cdots, z_n)$ of integers, we have $(z_1, \cdots, z_n) \in S$ iff $\phi(z_1, \cdots, z_n)$ is valid. A set S of n-tuples of integers is *linear*, iff there are a finite set $\vec{z}_1, \cdots, \vec{z}_n$ of n-tuples of integers s.t. $\vec{z} \in S$ iff $\vec{z} + \lambda_1\vec{z}_1 + \cdots + \lambda_n\vec{z}_n \in S$ for all integers λ_i where $1 \leq i \leq n$. A set S of n-tuples of integers is *semi-linear*, iff it is a finite union of linear sets. Ginsburg and Spanier [25] showed that semi-linear sets exactly captures LIA-definable sets[1].

Theorem 1 ([25]). *A set S of n-tuples of integers is LIA-definable iff S is semi-linear.*

[1]The original theorems in [25, 26] is stated over the natural numbers. However, it can be extended to the integers.

Cobham [26] established a fundamental connection between LIA-definable sets and periodicity of the set in one-dimensional case. A set S of integers is *ultimately periodic*, iff (1) there are two integers $n_l < 0$ and $p_l > 0$ s.t. for every $x \leq n_l$, $x \in S \cap \{z \mid z \leq n_l\}$ iff $x - p_l \in S \cap \{z \mid z \leq n_l\}$, and (2) there are two integers $n_u > 0$ and $p_u > 0$ s.t. for every $x \geq n_u$, $x \in S \cap \{z \mid z \geq n_u\}$ iff $x + p_u \in S \cap \{z \mid z \geq n_u\}$.

Theorem 2 ([26])**.** *A set S of integers is LIA-definable iff S is ultimately periodic.*

For example, the set of even integers has period 2 and is therefore LIA-definable, which can be represented by $x \equiv_2 0$. However, the set of prime numbers is not LIA-definable.

2.2. Partial maximum satisfiability

Given a propositional formula in conjunctive normal form, the *satisfiability (SAT)* problem aims to determine if there is an assignment that satisfies all of the clauses in the formula. The *maximum satisfiability (MaxSAT)* problem, the optimization variation of the SAT problem, searches a truth assignment that satisfies the maximum number of clauses. One of the significant generalizations of the MaxSAT problem is the *partial MaxSAT* problem. It divides the clauses into two types: *hard* and *soft*, and searches an assignment that satisfies all of the hard clauses and maximizes the number of satisfied soft clauses.

2.3. Impartial combinatorial games

In this section, we briefly introduce *impartial combinatorial games (ICGs)*. A game that satisfies the following conditions is called an ICG [4]: (1) There are two players and many states such that the player can move from one state to another one. (2) Two players alternate moving and have the same choice of moving. (3) The game ends when it moves to an *ending state* in which no player has a possible move. An *instance* of an ICG is the ICG together with an initial state. In general, an ICG may contain infinite states, and hence it may have

infinite instances. An ICG is *terminating*, iff each of its instances always ends in a finite number of moves. The problem of deciding if an ICG is terminating is undecidable [**?**]. Unless stated otherwise, we assume that ICGs are terminating in the remaining of this paper.

Each instance of an ICG is a finite two-player game with perfect information and no-chance move where the players take turns moving. In addition, any terminating ICG does not end in a draw. By Zermelo's Theorem [27], there always exists a winning strategy for one player in an ICG.

It can be easily observed that the player cannot guarantee to win in all states. To distinguish states in which the player can forces to win from those in which she cannot, we classify states into two types: *winning* and *losing states*.

Definition 1 ([4])**.** In an ICG, the sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are recursively defined as

1. All ending states are losing states.
2. All states such that there is at least one move to a losing state are winning states.
3. All states such that the only possible moves are to winning states are losing states.

Definition 1 provides the exact characteristics of winning and losing states in a recursive way. In the base case, Condition 1 requires all legal ending states to be losing states. Both Conditions 2 and 3 are induction steps. Condition 2 says that every state such that at least one applicable move results in a losing state is a winning state. Condition 3 means that every state such that all possible successor states are winning states is a losing state. The sets $\mathcal{W}$ of winning states and $\mathcal{L}$ of losing states are disjoint.

The above definition characterizes the concepts of losing states and winning states under the normal play rule. Under the misère rule, the definitions of winning and losing states are the same as Definition 1 except that all ending states are winning states. For the sake of clarity, throughout this paper, we assume that ICGs follow the normal play rule unless explicitly stated otherwise.

Our approach to constructing winning strategies can be easily lifted to the case under the misère rule and will be discussed in Section 6.

We hereafter introduce the concept of *strategy*. ICGs are a special case of Gale-Stewart games, and so are determined by memoryless strategies [28]. In addition, it is impossible to obtain a winning strategy for the player in a losing state, there is no need to consider how to choose a move in this case. We hence define a *strategy* as a function from winning states to moves. A strategy is *winning*, iff it is a strategy and every winning state leads to a losing state following the move defined in this strategy.

3. A logical framework

In this section, we first introduce the concepts of numeric states and actions, and then present a logical framework for describing ICGs based on numeric states and actions. The framework we adopt is a slight modification of the linear integer arithmetic fragment of the PDDL 2.1 language defined in [16].

3.1. Numeric states and actions

Throughout this paper, we use the notion $\mathcal{X}$ for the set of state variables that are used to define numeric states. An expression e (resp. a formula ϕ) is *semi-ground*, if $e \in \texttt{Exp}_{\mathcal{X}}$ (resp. $\phi \in \texttt{Form}_{\mathcal{X}}$). For example, suppose that $\mathcal{X} = \{v_1, v_2\}$. The expression $v_1 - 1$ is semi-ground, but $v_2 + k$ is not since k is not a state variable.

A *numeric state* s is an interpretation that maps every state variable v to an integer $v(s)$. For a state s, we can evaluate a semi-ground expression e into an integer $e(s)$ that the expression simplifies to when replacing each state variable v with its corresponding value $v(s)$. The Boolean value $\phi(s)$ of a semi-ground formula ϕ can be defined in a similar way. A numeric state s *satisfies* a formula ϕ, denoted by $s \models \phi$, iff $\phi(s) = \top$. A formula ϕ *implies* another one ψ, denoted by $\phi \models \psi$, iff $s \models \psi$ for every state s satisfying ϕ. A formula ϕ is *valid*, iff $s \models \phi$ for every state s; otherwise, it is *invalid*.

Each *numeric action* $a(\mathcal{K})$ is parameterized by a vector $\mathcal{K}$ of numeric variables, and is defined by a pair $\langle \texttt{pre}, \texttt{eff} \rangle$ where $\texttt{pre} \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$ denotes the precondition and $\texttt{eff}$ is the effect represented by a set of numeric effects. A *numeric effect* is a triple $\langle \phi, v, e \rangle$ where $\phi \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$, $v \in \mathcal{X}$, and $e \in \texttt{Exp}_{\mathcal{X} \cup \mathcal{K}}$. Intuitively, it means that if ϕ holds in the state s, then the value of v becomes $e(s)$ after executing the action; otherwise, it remains unchanged. Given an action $a(\mathcal{K})$, we use $\texttt{pre}(a(\mathcal{K}))$ and $\texttt{eff}(a(\mathcal{K}))$ for its precondition and effect, respectively.

Let $a(\mathcal{K})$ be an action and Σ a vector of semi-ground expressions where $\mathcal{K}$ and Σ have the same length n. The notation $e[\Sigma]$ (resp. $\phi[\Sigma]$) is the semi-ground expression (resp. formula) obtained from $e \in \exp_{\mathcal{X} \cup \mathcal{K}}$ (resp. $\phi \in \texttt{Form}_{\mathcal{X} \cup \mathcal{K}}$) by replacing the i-th parameter k_i of $\mathcal{K}$ with the i-th semi-ground expression e_i of Σ for $1 \leq i \leq n$. A *semi-ground action* $a[\Sigma]$ is defined as $\langle \texttt{pre}, \texttt{eff} \rangle$ where $\texttt{pre}(a[\Sigma]) = \texttt{pre}(a(\mathcal{K}))[\Sigma]$ and $\langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ for every $\langle \phi, v, e \rangle \in \texttt{eff}(a(\mathcal{K}))$. A semi-ground action $a[\Sigma]$ is *applicable* in a state s, iff $s \models \texttt{pre}(a[\Sigma])$. The result of applying an applicable action $a[\Sigma]$ in a state s is a successor state, written $\tau(s, a[\Sigma])$, where every variable v is associated to an integer $v(\tau(s, a[\Sigma]))$ defined as:

$$v(\tau(s, a[\Sigma])) = \begin{cases} e[\Sigma](s), & \text{if there is } \langle \phi[\Sigma], v, e[\Sigma] \rangle \in \texttt{eff}(a[\Sigma]) \text{ and } \phi[\Sigma](s) = \top; \\ v(s), & \text{otherwise.} \end{cases}$$

We require that every action $a(\mathcal{K})$ is not *self-contradictory*, formally, for every state s and every vector Σ of semi-ground expressions, if $\langle \phi_1[\Sigma], v_1, e_1[\Sigma] \rangle$, $\langle \phi_2[\Sigma], v_2, e_2[\Sigma] \rangle \in \texttt{eff}(a[\Sigma])$ s.t. $\phi_1[\Sigma](s) = \phi_2[\Sigma](s) = \top$, then $e_1[\Sigma](s) = e_2[\Sigma](s)$.

For brevity, we sometimes use a for (semi-ground) actions.

3.2. Formalization

Definition 2. An LIA-definable game is defined as a tuple $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ where

- $\mathcal{X}$: a finite set of state variables.

- $\mathcal{A}$: a finite set of numeric actions.
- $\mathcal{C}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all legal states.
- $\mathcal{E}$: a formula of $\texttt{Form}_{\mathcal{X}}$ denoting all ending states.

To obtain the finite representation of winning states, we use an arithmetic formula to exactly capture this notion, called *the winning formula*.

Definition 3. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. We say a formula $\phi \in \texttt{Form}_{\mathcal{X}}$ is a *winning formula* of Π, iff for every state s, $s \models \mathcal{C} \wedge \phi$ iff s is a legal winning state.

The winning formula ϕ along with the formula $\mathcal{C}$ exactly characterizes the set of winning states. In general, there are many logical different winning formulas for an ICG. But they are logically equivalent under the background theory $\mathcal{C}$, that is, $\mathcal{C} \models \phi \equiv \phi'$. Hence, we sometimes call a winning formula the winning one. Since the sets of winning states and losing states are disjoint, $\neg\phi$ is the symbolic representation of losing states. As proved in [29], the winning formulas of certain games are not LIA-definable. In this paper, we focus on the synthesis of LIA-definable winning formulas and leave the synthesis of winning formulas that require more expressive languages to future work.

A *rule* (ψ, a) is a pair of semi-ground formulas ψ and semi-ground actions a. Given a rule (ψ, a), we call ψ the *condition formula* of (ψ, a). A *strategy* δ is a set of rules.

Definition 4. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- A rule (ψ, a) is *winning*, iff for every state s satisfying $\mathcal{C} \wedge \phi \wedge \psi$, we have a is applicable in s and $\tau(s, a)$ is a losing state.
- A rule (ψ, a) is *complete*, iff for every state s s.t. $\tau(s, a)$ is a losing state, we have $\psi(s) = \top$.
- A rule (ψ_i, a_i) is *redundant* w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$, iff every state s satisfying ψ_i, there is a formula ψ_j s.t. $i \neq j$ and $\psi_j(s) = \top$.

- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *winning*, iff every rule (ψ_i, a_i) is winning.

- A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is *exhaustive*, iff for every winning state s of Π, there is a formula ψ_i s.t. $\psi_i(s) = \top$.

The winning property of the rule (ψ, a) requires that executing the action a in every state satisfying $\mathcal{C} \wedge \phi \wedge \psi$ yields a losing state. The completeness property of the rule says that the formula ψ covers all of the states s s.t. its successor state $\tau(s, a)$ is a losing state. The redundancy property of the rule (ψ_i, a_i) w.r.t. the strategy means that all condition formulas of the other rules of δ covers the condition formula of ψ_i. The winning property of the strategy means that each of its rule is a winning rule. The exhaustiveness property of the strategy stipulates that each winning state has its corresponding winning forced action.

Example 2. The formalization of the Take-away-3 game [11] is given as follows. The state variable v denotes the number of the remaining chips in this game. For simplicity, we use an integer c to denote the numeric state where v is assigned to c. A move consists of taking any number (from 1 to m) of chips. Here we consider the case where $m = 3$. The following are the definition of ICG and the description of actions.

- $\mathcal{X} : \{v\}$;
- $\mathcal{A} : \{take(k)\}$;
- $\mathcal{C} : v \geq 0$;
- $\mathcal{E} : v = 0$;
- $\texttt{pre}(take(k)) : v \geq k \wedge k > 0 \wedge k \leq 3$;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$.

The two numeric states 0 and 4 are legal losing states. The former is a legal ending state. The latter has three applicable semi-ground actions: $take(1)$,

Algorithm 1: `SynWinForm`

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an ICG.

Output: ϕ: the winning formula of Π

```
Initialize the set S of legal states
P, N ← ClassifyStates(S)
while true do
    φ ← GenerateCandidate(P, N)
    S' ← VerifyWinForm(φ)
    if S' = ∅ then return φ ;
    else
        P', N' ← ClassifyStates(S')
        P ← P ∪ P' and N ← N ∪ N'
```

$take(2)$, and $take(3)$. Executing each of them yields three different winning states $\tau(4, take(1)) = 3$, $\tau(4, take(2)) = 2$, and $\tau(4, take(3)) = 1$.

The winning formula of this game is $v \not\equiv_4 0$. It means that any legal state in which the number of the remaining chips is not a multiple of 4 is a winning state. The three rules $(v \equiv_4 1, take(1)), (v \equiv_4 2, take(2))$ and $(v \equiv_4 3, take(3))$ are winning and complete rules, and form a winning and complete strategy. But $(v = 2, take(2))$ is a winning but incomplete rule as there is a state 6 from which taking 2 chips leads to a losing state, which does not satisfy $v = 2$. □

4. Synthesizing winning formulas

In this section, we propose a counterexample-guided inductive synthesis (CEGIS) approach to synthesizing winning formulas for ICGs over infinite states, illustrated in Algorithm 1. A counterexample in this approach in fact is a state in ICGs.

4.1. The main algorithm

The main idea of Algorithm 1 is as follows: It first attempts to generate a candidate formula ϕ via a set of counterexamples, and then verifies the correctness of ϕ. If ϕ passes the verification, then the algorithm terminates with the winning formula ϕ. Otherwise, a set of new counterexamples will be gathered and used to create a new candidate formula. The above steps repeat until the algorithm discovers the winning formula of the ICG.

We hereafter elaborate Algorithm 1 in details. It initializes the set S as a finite set of legal states, which is accomplished by an SMT solver (Line 1). Each legal state s is the state satisfying the formula $\mathcal{C}$ denoting all legal states (cf. Definition 2). The SMT solver takes the formula $\mathcal{C}$ as input, and returns a state satisfying $\mathcal{C}$. However, the SMT solver often returns the same state for the formula $\mathcal{C}$. We can easily fix this defect and generate many different legal states via adding an extra subformula on $\mathcal{C}$. Suppose that we are given a set S' of legal states. The formula $\mathcal{C}_{S'}$ is $\mathcal{C} \wedge \bigwedge_{s \in S'} \neg[\bigwedge_{v \in \mathcal{X}}(v = v(s))]$, and hence any state satisfying $\mathcal{C}_{S'}$ is a legal state not in S'.

The set S of states will be classified into two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states (Line 2). In an ICG, each positive state is a winning state while each negative state is a losing state.

Algorithm 1 then enters a loop until a winning formula is discovered (Lines 3 - 9). With the two sets of positive states and of negative states in hand, we can produce a candidate winning formula ϕ (Line 4). Then, we will check whether the candidate formula ϕ is a winning formula (Line 5). If ϕ is correct (that is, $S' = \emptyset$), then Algorithm 1 terminates with the solution ϕ (Line 6). Otherwise, the verification procedure yields some states of S' that are witnesses of its failure and incorporates them into $\mathcal{P}$ and $\mathcal{N}$, respectively (Lines 8 - 9).

Example 3. Table 1 shows the run of synthesizing the winning formula of Takeaway-3 game. At the beginning, only state 1 is in the set $\mathcal{P}$ and state 0 in $\mathcal{N}$. Our approach generates the formula $v = 1$ as the candidate solution that is consistent with $\mathcal{P}$ and $\mathcal{N}$. However, it is not the correct winning formula that holds for all

$\mathcal{P}$	$\mathcal{N}$	Candidate formulas	Counterexamples
1	0	$v = 1$	2
1, 2	0	$v > 0$	4
1, 2	0, 4	$v > 0 \wedge v < 3$	3
1, 2, 3	0, 4	$v > 0 \wedge v \leq 3$	5
1, 2, 3, 5	0, 4	$v \not\equiv_4 0$	$\emptyset$

Table 1: A run of synthesizing the winning formula

the legal states and the SMT solver returns a winning state 2. Our approach takes into account the new state. Three candidate formulas, $v > 0$, $v > 0 \wedge v < 3$, and $v > 0 \wedge v \leq 3$, are found sequentially. However, all three formulas fail verification. The corresponding counterexamples are the losing state 4 and the winning states 3 and 5. Finally, the algorithm terminates with the correct winning formula $v \not\equiv_4 0$, which is constructed from the above 6 states. □

Finally, three subprocedures of Algorithm 1 remain to be addressed: (1) classifying winning states from losing states (Line 2), (2) generating a candidate formula (Line 4), and (3) verifying the correctness of a candidate formula (Line 5). These will be resolved in the subsequent three sections.

4.2. Classification of numeric states

In this subsection, we design two algorithms to the problem of classifying states into winning and losing states. Both algorithms work in a recursive way based on Definition 1.

We first provide the notion of *transition formulas* and *global transition formulas*. The definition of transition formulas is a numerical generalization of the existing propositional encoding for the planning problem which focuses on transition systems over finite states [30, 31]. The transition formula $\mathcal{T}(a(\mathcal{K}))$ for an action $a(\mathcal{K})$ reflects the mapping from predecessor states to successor states due to the effect of $a(\mathcal{K})$. To distinguish the predecessor state variable and the successor one, we use the unprimed variable v for the former and the primed one v' for the latter.

Definition 5. The transition formula $\mathcal{T}(a(\mathcal{K}))$ of $a(\mathcal{K})$ is the conjunction of the following formulas:

1. $\texttt{pre}(a(\mathcal{K}))$;
2. $\bigwedge_{\langle\phi,v,e\rangle\in\texttt{eff}(a(\mathcal{K}))}(\phi \rightarrow v' = e)$;
3. $\bigwedge_{v\in\mathcal{V}}[(\bigvee_{\langle\phi,v,e\rangle\in\texttt{eff}(a(\mathcal{K}))}\phi) \vee v' = v]$.

The first formula is the precondition of action $a(\mathcal{K})$. The second one is the effect axiom meaning that the value of v becomes $e(s)$ if the condition ϕ holds in the state s. The third formula is the frame axiom meaning that if no condition ϕ s.t. $\langle\phi, v, e\rangle \in \texttt{eff}(a)$ holds, then the value of v remains unchanged.

The global transition formula $\mathcal{T}(\mathcal{A})$ is the union of the mapping due to the effect of every action $a(\mathcal{K}) \in \mathcal{A}$, denoting the state transition system of the whole game. The definition is as follows:

Definition 6. Let $\mathcal{A}$ be a set of actions. The global transition formula $\mathcal{T}(\mathcal{A})$ of Π is $\bigvee_{a(\mathcal{K})\in\mathcal{A}} \exists\mathcal{K}[\mathcal{T}(a(\mathcal{K}))]$.

The following example illustrates how transition formulas are constructed from the definition of actions.

Example 4. Let us continue the Take-away-3 game. We construct the transition formula $\mathcal{T}(take(k))$ as follows. The precondition is $v \geq k \wedge k > 0 \wedge k \leq 3$. The effects contain one elements: $\langle\top, v, v - k\rangle$. Hence, the effect axiom of $\mathcal{T}(take(k))$ is $(\top \rightarrow v' = v-k)$. We now consider the frame axiom of $\mathcal{T}(take(k))$. For variable v, the formula is $\top \vee v' = v$. Conjoining the above formulas and simplifying the conjunction lead to the transition formula:

$$\mathcal{T}(take(k)) \equiv v \geq k \wedge k > 0 \wedge k \leq 3 \wedge v' = v - k.$$

By Definition 6, the global transition formula

$$\mathcal{T}(\mathcal{A}) \equiv \exists k[\mathcal{T}(take(k))].$$

□

Algorithm 2: `IsWinningState`(Π, ϕ, s)

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of Π;

s: a legal state.

Output: $\top$, if s is a winning state; $\bot$, otherwise.

```
1  if Cache(s) ≠ nil then return Cache(s) ;
2  if s ⊨ C ∧ E then
3  |  Cache(s) ← ⊥
4  |  return ⊥
5  foreach legal successor state s' of s do
6  |  if IsWinningState(Π, φ, s') = ⊥ then
7  |  |  Cache(s) ← ⊤
8  |  |  return ⊤
9  Cache(s) ← ⊥
10 return ⊥
```

Algorithm 2 provides a recursive procedure for determining whether a legal state s in an LIA-definable ICG is a winning state. The algorithm employs memorization to avoid redundant computation by first checking if the result for state s has already been cached (Line 1). It then verifies if s is a legal ending state (Lines 2 - 4). If it is the case, then s is classified as a losing states since all ending states are losing states (cf. Definition 1). Otherwise, Algorithm 2 iterates over every legal successor state s' of s (Lines 5 - 8). If at least one successor state s' is a losing state, then s is a winning state since all states such that there is at least one move to a losing state are winning states. Conversely, s is a losing state since all states such that the only possible moves are to winning states are losing states (Lines 9 - 10).

The following example illustrates the run of Algorithm 2.

Example 5. Let us consider the Take-away-3 game with initial state 4. Figure

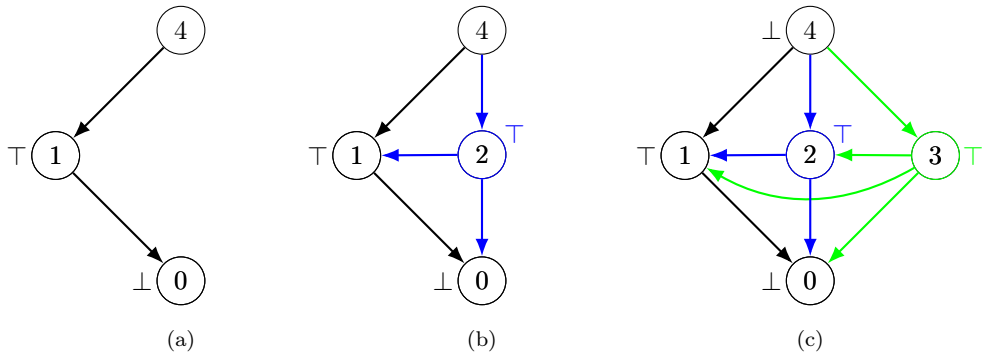


Figure 1: The run of Algorithm 2 in Take-away-3 game.

1 illustrates a run of Algorithm 2 on this game. Since state 4 is not in the cache and is not an ending state, the algorithm recursively explores its successor states 1, 2, and 3. It first visits state 1, whose unique successor is state 0. Because state 0 is an ending state, it is classified as losing and stored in the cache. Consequently, state 1 is classified as winning and stored in the cache, as shown in Figure 1(a). The algorithm then evaluates state 2. Since state 2 has a successor state 0, which is already known to be losing, state 2 is classified as winning, as shown in Figure 1(b). Similarly, state 3 is classified as winning because it also has a move to state 0, as shown in Figure 1(c). As all successor states of state 4 are winning, state 4 itself is classified as losing. □

4.3. Generation of candidate formulas

In this subsection, we propose two approaches for generating candidate formulas. The first approach adopts a simple enumerative algorithm proposed in [17]. The second approach formulate the task as exact binary classification problem and encode each classification problem as an instance of the partial MaxSAT problem.

4.3.1. Enumeration-based approach

The key to the enumeration algorithm is to process generate the candidate formulas (or expressions) by induction on size according to the grammar illus-

trated in Section 2.1. In the first step, only arithmetic expressions of size 1 (*i.e.*, state variables and integers) are considered. As the set of integers is infinite, the integers considered in the first step are 0, 1 together with the constants explicitly occurring in the formalization of games. Firstly, This restriction does not limit the coverage of all integers since any positive integer c can be expressed as the sum of c 1's, while the negative integer $-c$ equals to $0 - c$. Furthermore, we observe that some integers appearing in the winning formula and strategy are close to some explicit constants defined in the formalization. Hence, supplementing these constants can accelerate the synthesis process. In the k-th step, we generate arithmetic expressions and formulas of size k. The enumeration process terminates when it finds candidates formula ϕ such that $\{s \mid s \models \phi \wedge \mathcal{C}\} = \mathcal{P}$.

To improve the efficiency of the enumeration process, we remove expressions and formulas based on the notion of indistinguishability. Formally, two formulas ϕ_1 and ϕ_2 are *indistinguishable* w.r.t. a set S of states, if $\phi_1(s) = \phi_2(s)$ for every $s \in S$. The definition of indistinguishability between two expressions is similar. For example, suppose that $S = \{(v_1 : 1, v_2 : 2), (v_1 : 2, v_2 : 2)\}$. It is easily verified that the two formulas $v_1 \neq v_2$ and $v_1 + 1 = v_2$ hold in the first state and does not hold in the second one. Therefore, the two formulas are indistinguishable w.r.t. S. Suppose that we are generating expressions and formulas of size k. If ϕ_1 and ϕ_2 are indistinguishable w.r.t. S, then it is sufficient to carry forward only one of ϕ_1 and ϕ_2 to the next iteration $k + 1$. Based on the notion of indistinguishability, we can prune the search space of candidate formulas.

4.3.2. Exact binary classification-based approach

We observe that enumeration-based approach does not scale well since exhaustive generation of all candidate formulas becomes time consuming when the size of the winning formula increases. To accelerate the generation process, we introduce the exact binary classification problem and propose an approach that encodes this problem as a partial MaxSAT problem.

In contrast to enumeration-based approach, we firstly generates a set Ψ of arithmetic atoms rather than formulas of increasing sizes until for every pair

of positive state and negative state, there is an atom $\psi \in \Psi$ classifying them. In order to prune the huge size of Ψ, we also eliminate some arithmetic atoms in Ψ based on indistinguishability. Based on the set Ψ of arithmetic atoms, we reduce the problem of searching for candidate formulas to the exact binary classification problem.

Definition 7. Given two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and a set Ψ of arithmetic atoms, the task of the exact binary classification problem is to generate a formula ϕ based on Ψ s.t. $\phi(s) = \top$ for every state $s \in \mathcal{P}$ and $\phi(s) = \bot$ for every state $s \in \mathcal{N}$.

The exact binary classification problem may not have a solution in some cases. The sufficient and necessary condition for the existence of a solution is established by the following theorem.

Theorem 3. *Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The following two statements are equivalent:*

1. *For each pair of positive state s and negative state t, there is an atom $\psi \in \Psi$ s.t. $\psi(s) \neq \psi(t)$.*
2. *There exists a formula ϕ based on Ψ that classifies $\mathcal{P}$ from $\mathcal{N}$.*

To prove the above theorem, we need the following two definitions that are used to synthesize a desired formula for the exact binary classification problem if the solution exists.

Definition 8. Let $\mathcal{P}$ and $\mathcal{N}$ be two sets of positive and negative states, respectively, and Ψ a set of atoms. The *filter set* $\Psi_{\mathcal{P},\mathcal{N}}$ of Ψ w.r.t. $\mathcal{P}$ and $\mathcal{N}$ is $\{\psi \mid \psi \in \Psi \text{ and } \psi(s) \neq \psi(t) \text{ for some } s \in \mathcal{P} \text{ and } t \in \mathcal{N}\}$.

The filter set is the subset of Ψ in which each element classifies at least one positive state and one negative state.

Definition 9. Let S be a set of states and Ψ a set of atoms. The *characteristic formula* of S based on Ψ is $\bigvee_{s \in S} \left[\left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \top} \psi \right) \wedge \left(\bigwedge_{\psi \in \Psi \text{ and } \psi(s) = \bot} \neg \psi \right) \right]$.

The conjunct $\left(\bigwedge_{\psi\in\Psi \text{ and } \psi(s)=\top} \psi\right) \wedge \left(\bigwedge_{\psi\in\Psi \text{ and } \psi(s)=\bot} \neg\psi\right)$ can be considered as a truth assignment of the state s on the set Ψ of atoms. The characteristic formula collects the assignments of every state $s \in S$ on Ψ.

Proof of Theorem 3: ($\Leftarrow$): Let ϕ be a formula based on Ψ s.t. ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. We prove the first statement by contradiction. Let $s \in \mathcal{P}$ and $t \in \mathcal{N}$ s.t. no atom $\psi \in \Psi$ satisfies the property $\psi(s) \neq \psi(t)$. It follows that $\phi(s) = \phi(t)$. This contradicts the assumption.

($\Rightarrow$): Let ϕ be the characteristic formula of $\mathcal{P}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$. Clearly, ϕ is also based on Ψ. By Definition 9, for every positive state s, ϕ contains $\left(\bigwedge_{\psi\in\Psi \text{ and } \psi(s)=\top} \psi\right) \wedge \left(\bigwedge_{\psi\in\Psi \text{ and } \psi(s)=\bot} \neg\psi\right)$. Hence, $s \models \phi$. By the first statement of Theorem 3 and Definition 8, for every negative state t, there is an atom $\psi \in \Psi_{\mathcal{P},\mathcal{N}}$ and a positive state s s.t. $\psi(t) \neq \psi(s)$. None of the conjuncts of ϕ is satisfied by t. Hence, $t \not\models \phi$. In summary, ϕ classifies $\mathcal{P}$ from $\mathcal{N}$. □

The proof of Theorem 3 offers a direct method to obtain a formula that discriminates positive states from negative ones. However, the given set Ψ in general includes a large number of arithmetic atoms. So is the filter set $\Psi_{\mathcal{P},\mathcal{N}}$. Therefore, the characteristic formula of $\mathcal{P}$ based on $\Psi_{\mathcal{P},\mathcal{N}}$ has large size and is difficult to be applied in generating candidate formulas.

To decrease the amount of atoms in the solution, we devise a practical method to the exact binary classification problem based on partial MaxSAT. The propositional encoding $T(\mathcal{P},\mathcal{N},\Psi)$ takes as inputs two sets $\mathcal{P}$ of positive states and $\mathcal{N}$ of negative states and the set Ψ of arithmetic atoms. The propositions in $T(\mathcal{P},\mathcal{N},\Psi)$ are

- $Sel(\psi)$: the atom $\psi \in \Psi$ appears in the formula ϕ.

The hard constraints of $T(\mathcal{P},\mathcal{N},\Psi)$ are

- $\bigvee_{\psi(s)\neq\psi(t)} Sel(\psi)$ for every $s \in \mathcal{P}$ and $t \in \mathcal{N}$.

It means that for every pair of positive state s and negative state t, at least one atom of Ψ that distinguishes between s and t is chosen, corresponding to the first statement of Theorem 3.

States		$v>0$	$v>1$	$v>2$	$v>3$	$v>4$	$v=0$	$v=1$	$v=2$	$v=3$	$v=4$
$\mathcal{P}$	1	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$
	2	$\top$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$
	3	$\top$	$\top$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$
	5	$\top$	$\top$	$\top$	$\top$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$
$\mathcal{N}$	0	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$
	4	$\top$	$\top$	$\top$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$

States		$v\equiv_2 0$	$v\equiv_2 1$	$v\equiv_3 0$	$v\equiv_3 1$	$v\equiv_3 2$	$v\equiv_4 0$	$v\equiv_4 1$	$v\equiv_4 2$	$v\equiv_4 3$
$\mathcal{P}$	1	$\bot$	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$
	2	$\top$	$\bot$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$
	3	$\bot$	$\top$	$\top$	$\bot$	$\bot$	$\bot$	$\bot$	$\bot$	$\top$
	5	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$	$\top$	$\bot$	$\bot$
$\mathcal{N}$	0	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$
	4	$\top$	$\bot$	$\bot$	$\top$	$\bot$	$\top$	$\bot$	$\bot$	$\bot$

Table 2: The example of learning the characteristic formula

The clauses in the soft constraints of $T(\mathcal{P},\mathcal{N},\Psi)$ are the negative literal $\neg Sel(\psi)$ for each atom ψ. The partial MaxSAT solver looks for a solution with the fewest possible arithmetic atoms.

Example 6. We use Take-away-3 game to illustrate the method to learn the characteristic formula. Suppose that we are given the sets of positive states $\mathcal{P}:\{1,2,3,5\}$ and of negative states $\mathcal{N}:\{0,4\}$, and the set of arithmetic atoms

$$\Psi=\left\{\begin{array}{l} v>0, v>1, v>2, v>3, v>4,\\ v=0, v=1, v=2, v=3, v=4,\\ v\equiv_2 0, v\equiv_2 1,\\ v\equiv_3 0, v\equiv_3 1, v\equiv_3 2,\\ v\equiv_4 0, v\equiv_4 1, v\equiv_4 2, v\equiv_4 3 \end{array}\right\}.$$

The Boolean value of every atom on every state is shown in Table 2. The set Ψ contains 19 atoms, so the propositional encoding $T(\mathcal{P},\mathcal{N},\Psi)$ includes 19 propositions, one for each atom in Ψ. Since nine atoms distinguish positive state 1 and negative state 0, the hard constraint for 1 and 0 is $Sel(v>0)\vee Sel(v=$

$0) \vee Sel(v = 1) \vee Sel(v \equiv_2 0) \vee Sel(v \equiv_2 1) \vee Sel(v \equiv_3 0) \vee Sel(v \equiv_3 1) \vee Sel(v \equiv_4 0) \vee Sel(v \equiv_4 1)$. The hard constraints for other pairs of positive state and negative state can be similarly constructed. The soft constraints are the negations of these 19 propositions. The partial MaxSAT solver returns an assignment in which only $Sel(v \equiv_4 0)$ is $\top$ and the other 18 propositions are $\bot$.

Only the atom $v \equiv_4 0$ is picked from the assignment. As the chosen atom does not hold in every state of $\mathcal{P}$, its negation $v \not\equiv_4 0$ is the characteristic formula of $\mathcal{P}$ based on Ψ. In addition, it exactly classifies the given states of positive states and of negative states. □

4.4. Correctness verification

In this subsection, we present the constraints for the winning formula that is used to verifying the correctness of candidate formula based on the notion of global transition formulas (cf. Definition 6).

Definition 10. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The constraints for the winning formula ϕ of Π are as follows:

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$;
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$;
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$.

where $\psi[\mathcal{X}/\mathcal{X}']$ is the formula obtained by replacing every occurrence of $v \in \mathcal{X}$ in ψ with v'.

Each of the above constraints corresponds to each condition of winning and losing states (cf. Definition 1). As a result, we obtain the soundness theorem of the constraints for the winning formula.

Theorem 4. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. A formula ϕ is the winning formula of Π iff all of the constraints illustrated in Definition 10 are valid.*

Proof. ($\Rightarrow$): Assume that ϕ is the winning formula. It follows that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing. We prove the three constraints illustrated in Definition 10 are valid.

1. $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$: Let s be a state satisfying $\mathcal{C} \wedge \mathcal{E}$. It follows that s is a legal ending state. By the first item of the definition of winning and losing states (cf. Definition 1), s is a losing state. Hence, $s \models \neg\phi$ and the first constraint is valid.
2. $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$: Let s be a state satisfying $\mathcal{C} \wedge \phi$. It follows that s is a legal winning state. By the second item of the definition of winning and losing states, there is at least one applicable action a in s s.t. $\tau(s, a)$ is a losing state . Hence, $s \models \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$ and the second constraint is valid.
3. $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$: The proof of the validity of the third constraint is the same as that of the second constraint by using the third item of the definition of winning and losing states.

($\Leftarrow$): Assume that ϕ is a formula s.t. all of the constraints illustrated in Definition 10 is valid. We define a relation $\prec$ between states in the following:

$$s' \prec s \text{ iff } s' \text{ is a successors state of } s.$$

Since Π is terminating, the game always ends in a finite number of moves from any legal state. Hence, $\prec$ contains no infinite descending chains and is well-founded. Let S be the legal states of Π. It follows that $(S, \prec)$ is a well-founded set. We prove that every state s satisfying $\mathcal{C} \wedge \phi$ is winning and every state s satisfying $\mathcal{C} \wedge \neg\phi$ is losing by induction on $(S, \prec)$.

Base case (s is a legal ending state): It follows that $s \models \mathcal{C} \wedge \mathcal{E}$. By the first constraint $\mathcal{C} \wedge \mathcal{E} \rightarrow \neg\phi$, we get that $s \models \neg\phi$. By the definition of winning formula (cf. Definition 3), s is a losing state.

Inductive step (s is a legal but not ending state): Suppose that $s \models \phi$. By the second constraint $(\mathcal{C} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge (\mathcal{C} \wedge \neg\phi)[\mathcal{X}/\mathcal{X}']]$, there is a successor state $\tau(s, a)$ of s s.t. $\tau(s, a) \models \mathcal{C} \wedge \neg\phi$. By the inductive assumption, $\tau(s, a)$ is a losing state. By the second item of the definition of winning and losing states, s is a winning state. Similarly, we can prove that s is a losing state when $s \models \neg\phi$ by using the third constraint $(\mathcal{C} \wedge \neg\phi) \rightarrow \forall \mathcal{X}'[\mathcal{T}(\mathcal{A}) \rightarrow (\mathcal{C} \wedge \phi)[\mathcal{X}/\mathcal{X}']]$ and the third item of the definition of winning and losing states. □

Theorem 4 reduces the correctness verification problem of an LIA-formula for an LIA-definable ICG to the validity problem of an LIA-formula. Thanks to the decidability of the validity problem of LIA [32], the above correctness verification problem is decidable.

Corollary 1. *The problem deciding if an LIA-formula is a winning formula for an LIA-definable ICG is decidable.*

In addition, Theorem 4 confirms the decidability of correctness verification of an LIA-formula and the soundness property of Algorithm 1 for synthesizing the winning formula.

Corollary 2. *Let* $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ *be an ICG. The formula returned by Algorithm 1 is the winning formula of* Π.

Proof sketch: Lines 5 of Algorithm 1 checks if candidate formula ϕ is the winning formula according to the constraints illustrated in Definition 10. If $S' = \emptyset$, then the constraints for the winning formula are valid. By Theorem 4, ϕ is the winning formula. □

5. Synthesizing winning strategies

In this section, we also devise a CEGIS approach to synthesizing winning and exhaustive strategies. We first introduce the main algorithm in Algorithm 3, and then present the correctness verification of rules and strategies.

5.1. The main algorithm

The main synthesis algorithm first generate a pool of candidate actions for a set of positive states. For each action, we synthesize its condition formula via the candidate formula generation approach. Each action and its corresponding condition formula constitute a rule and a set of rules forms a strategy. Then, we check against the winning and completeness property of each rule and the winning and exhaustiveness property of the strategy. This step will continue

Algorithm 3: `SynWinStrat`$(\phi, \mathcal{P}, \mathcal{N})$

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

ϕ: the winning formula of the Π;

$\mathcal{P}$: the set of positive states;

$\mathcal{N}$: the set of negative states.

Output: δ: a winning and exhaustive strategy

$\delta^c, \delta^w, \mathcal{A}^f, \mathcal{A}^u \leftarrow \emptyset$

$\mathcal{P}' \leftarrow \mathcal{P}$

while true do

  $\mathcal{P}'' \leftarrow \mathcal{P}'$ and $\mathcal{P}' \leftarrow \emptyset$

  $\mathcal{A}' \leftarrow$ `EnumerateActions`$(\mathcal{P}'', \mathcal{A}^f \cup \mathcal{A}^u)$

  foreach $a \in \mathcal{A}^u \cup \mathcal{A}'$ **do**

    if $a \notin \mathcal{A}^u$ **then**

      $\mathcal{P}_a \leftarrow \{s \in \mathcal{P} \mid s \models \texttt{pre}(a)$ and $\tau(s, a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N} \cup \{s \in \mathcal{P} \mid s \not\models \texttt{pre}(a)$ or $\tau(s, a) \notin \mathcal{L}\}$

    else `/* `$a \in \mathcal{A}^u$` */`

      $\mathcal{P}_a \leftarrow \mathcal{P}_a \cup \{s \in \mathcal{P}'' \mid s \models \texttt{pre}(a)$ and $\tau(s, a) \in \mathcal{L}\}$

      $\mathcal{N}_a \leftarrow \mathcal{N}_a \cup \{s \in \mathcal{P}'' \mid s \not\models \texttt{pre}(a)$ or $\tau(s, a) \notin \mathcal{L}\}$

    $\psi_a \leftarrow$ `GenerateCandidate`$(\mathcal{P}_a, \mathcal{N}_a)$

    $S^w_a \leftarrow$ `VerifyWinningRule`(ψ_a, a)

    if $S^w_a \neq \emptyset$ **then**

      $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$ and $\mathcal{P}' \leftarrow \mathcal{P}' \cup S^w_a$

    else

      $S^c_a \leftarrow$ `VerifyCompleteRule`(ψ_a, a)

      if $S^c_a = \emptyset$ **then**

        $\mathcal{A}^f \leftarrow \mathcal{A}^f \cup \{a\}$ and $\mathcal{A}^u \leftarrow \mathcal{A}^u \setminus \{a\}$

        $\delta^c \leftarrow \delta^c \cup \{(\psi_a, a)\}$ and $\delta^w \leftarrow \delta^w \setminus \{(\psi'_a, a)\}$ `/* `(ψ'_a, a)` is the existing incomplete but winning rule of `δ^w` */`

      else

        $\mathcal{A}^u \leftarrow \mathcal{A}^u \cup \{a\}$ and $\mathcal{P}' \leftarrow \mathcal{P}' \cup S^c_a$

        $\delta^w \leftarrow (\delta^w \setminus \{(\psi'_a, a)\}) \cup \{(\psi_a, a)\}$

  $S^e \leftarrow$ `VerifyExhaustiveStrategy`$(\delta^c \cup \delta^w)$

  if $S^e = \emptyset$ **then return** `Simplify`$(\delta^c \cup \delta^w)$;

  else $\mathcal{P}' \leftarrow (\mathcal{P}' \cup S^e) \setminus \mathcal{P}$ and $\mathcal{P} \leftarrow \mathcal{P} \cup \mathcal{P}'$;

until we synthesize a winning and exhaustive strategy. The following gives a comprehensive explanation of Algorithm 3.

After the process of synthesizing winning formulas, we obtain the winning formula ϕ along with two sets $\mathcal{P}$ of winning states and $\mathcal{N}$ of losing states, which serve as inputs to Algorithm 3. Algorithm 3 maintains the following entities. δ^c is a set of winning and complete rules, while δ^w is a set of winning but incomplete rules. $\mathcal{A}^f$ contains actions a whose corresponding rule (ψ_a, a) has been verified to be both winning and complete. The verification of the winning and completeness properties will be elaborated in Section 5.2. $\mathcal{A}^u$ contains actions a whose corresponding rule is neither winning nor complete. $\mathcal{P}$ is the set of all generated winning states. It is initialized as the set of winning states produced by synthesizing the winning formula and is incrementally expanded during strategy synthesis. $\mathcal{P}'$ and $\mathcal{P}''$ are the sets of the newly discovered winning states at the current and previous iterations, respectively.

At the beginning, Algorithm 3 initializes the data items: δ^c, δ^w, $\mathcal{A}^f$, $\mathcal{A}^u$ and $\mathcal{P}'$ (Lines 1 - 2). It then enters a loop (Lines 3 - 27). At each iteration, it first generates a pool $\mathcal{A}'$ of new actions not in $\mathcal{A}^f \cup \mathcal{A}^u$ for $\mathcal{P}''$, which will be elaborated in Algorithm 4 (Line 5). For each state $s \in \mathcal{P}''$, the algorithm first checks whether some action in $\mathcal{A}^f \cup \mathcal{A}^u$ is applicable in s and leads to a losing state. If so, then no additional action is enumerated for s; otherwise, the algorithm search a new action for s. For every action $a \in \mathcal{A}^u \cup \mathcal{A}'$, Algorithm 3 tries to generate a desired condition formula ψ_a for a, which was introduced in Section 4.3 (Lines 6 - 24).

We collect the set $\mathcal{P}_a$ of positive states and $\mathcal{N}_a$ of negative states for action a (Lines 7 - 12). The set $\mathcal{P}_a$ is a subset of $\mathcal{P}$ s.t. each state $s \in \mathcal{P}$ satisfies the following conditions: (1) the action a is applicable in s; and (2) the successor state $\tau(s, a)$ is losing. In contrast, the set $\mathcal{N}_a$ includes all states in $\mathcal{N}$ together with those states in $\mathcal{P}$ that do not satisfy the above two conditions. If action a is a new action, then both $\mathcal{P}_a$ and $\mathcal{N}_a$ are constructed from $\mathcal{P}$ and $\mathcal{N}$ (Lines 7 - 9). Otherwise, $\mathcal{P}_a$ and $\mathcal{N}_a$ have been constructed before and are enlarged only with the states in $\mathcal{P}''$ (Lines 10 - 12).

The algorithm then generates a candidate formula ψ_a that separates $\mathcal{P}_a$ from

$\mathcal{N}_a$ via the two methods proposed in Section 4.3 (Line 13). The correctness of the rule (ψ_a, a) will be validated (Lines 14 - 24). We first verify the winning property of (ψ_a, a) (Line 14). If (ψ_a, a) is not winning, we are given a set S_a^w of states certifying that ψ_a is not a desired formula for the action a. Each state $s \in S_a^w$ has the following two properties: (1) s is a winning state; and (2) either a is not applicable in s, or $\tau(s, a)$ is not a losing state. Then, we add a into $\mathcal{A}^u$ and enlarge $\mathcal{P}'$ by S_a^w (Line 16). Otherwise, (ψ_a, a) is winning and we will examine its completeness property. This is because, in some cases, action a is the unique action that changes some winning states to a losing state, but ψ_a may not involve these winning states. If (ψ_a, a) is complete, then we add a into $\mathcal{A}^f$ and remove it from $\mathcal{A}^u$ (Line 20). At the same time, we add the rule (ψ_a, a) into δ^c and remove the existing incomplete but winning rule (ψ'_a, a) from δ^w (Line 21). Otherwise, we obtain a set S_a^c of states certifying that ψ_a is not a desired formula for the action a. Then, we handle this case similarly to the case where (ψ_a, a) is not winning (Line 23). In addition, we update the condition formula of the existing rule (ψ'_a, a) in δ^w to ψ_a (Line 24).

Finally, Algorithm 3 aggregate all rules of δ^c and δ^w into a strategy δ. If δ is exhaustive, then we terminate with a simplified strategy of δ^c and δ^w. The simplification process will repeatedly remove redundant rules of the strategy until no further rule can be removed. Otherwise, Algorithm 3 moves to the next iteration with two updated sets $\mathcal{P}'$ and $\mathcal{P}$ (Line 27).

Algorithm 4 takes the set $\mathcal{P}$ of winning states together with the set $\mathcal{A}^e$ of existing semi-ground actions as input, and outputs a pool $\mathcal{A}'$ of new semi-ground actions s.t. every state $s \in \mathcal{P}$ has its associated action $a[\Sigma] \in \mathcal{A}^e \cup \mathcal{A}'$ that is applicable in s and that leads s to a losing state. Algorithm 4 maintains a set Ω of semi-ground terms so as to enumerate semi-ground actions. For each state $s \in \mathcal{P}$, it first checks if $\mathcal{A}^e \cup \mathcal{A}'$ has the associated action $a[\Sigma]$ for s (Line 3). If so, the algorithm does not enumerate any action for s (Line 4). Otherwise, it considers each action $a(\mathcal{K}) \in \mathcal{A}$ and searches for a vector of parameters Σ based on Ω such that $s \models \texttt{pre}(a[\Sigma])$ and $\tau(s, a[\Sigma]) \in \mathcal{L}$ (Lines 5 - 15). Once such vector is found, the corresponding action $a[\Sigma]$ is added to $\mathcal{A}'$ (Line 9 -12). If

Algorithm 4: `EnumerateActions`$(\mathcal{P}, \mathcal{A}^{\mathtt{f}}, \mathcal{A}^{\mathtt{u}})$

Input: $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$: an LIA-definable ICG;

$\mathcal{P}$: the set of positive states;

$\mathcal{A}^e$: the set of existing actions;

Output: $\mathcal{A}'$: the pool of semi-ground actions.

```
1  Ω ← FindFirstTerm(P) and A' ← ∅
2  foreach s ∈ P do
3      if ∃a[Σ] ∈ A^e ∪ A' s.t. s ⊨ pre(a) and τ(s, a[Σ]) ∈ L then
4          continue;
5      Found ← ⊥
6      repeat
7          foreach a ∈ A do
8              foreach Σ ∈ Ω^k s.t. a[Σ] ∉ A^e ∪ A' do        /* k is the
                length of the parameter vector of a */
9                  if s ⊨ pre(a[Σ]) and τ(s, a[Σ]) ∈ L then
10                     A' ← A' ∪ {a[Σ]}
11                     Found ← ⊤
12                     break;
13             if Found then break;
14         Ω ← Ω ∪ FindNextTerm(P)
15     until Found;
16 return A'
```

no desired semi-ground action is generated for s, then the set Ω of semi-ground terms will be enlarged (Line 14).

Example 7. Table 3 shows the synthesis process of the winning strategy for the Take-away-3 game under the algorithm 3. According to example, we have the set of winning states $\mathcal{P} = \{1, 2, 3, 5\}$ and of losing states $\mathcal{N} = \{0, 4\}$.

In the first iteration, we enumerate the set of actions $\mathcal{A}' = \{take(1), take(2), take(3)\}$.

Iter.	$\mathcal{P}$	$\mathcal{A}'$	a	$\mathcal{P}_a$	$\mathcal{N}_a$	ψ_a	S_a^w	S_a^c	$\mathcal{A}^f$	$\mathcal{A}^u$	S^e
1	$1,2,3,5$	$take(1)$ $take(2)$ $take(3)$	$take(1)$	$1,5$	$0,2,3,4$	$v \equiv_4 1$	$\emptyset$	$\emptyset$	$take(1)$	$take(2)$ $take(3)$	6
			$take(2)$	2	$0,1,3,4,5$	$v = 2$	$\emptyset$	6			
			$take(3)$	3	$0,1,2,4,5$	$v = 3$	$\emptyset$	7			
2	$6,7$	$\emptyset$	$take(2)$	$2,6$	$0,1,3,4,5,7$	$v \equiv_4 2$	$\emptyset$	$\emptyset$	$take(1)$ $take(2)$ $take(3)$	$\emptyset$	$\emptyset$
			$take(3)$	$3,7$	$0,1,2,4,5,6$	$v \equiv_4 3$	$\emptyset$	$\emptyset$			

Table 3: A run of synthesizing the winning strategy

For action $take(1)$, the set $\mathcal{P}_{take(1)}$ of positive states for the condition formula is $\{1,5\}$ while the set $\mathcal{N}_{take(1)}$ of negative states is $\{0,2,3,4\}$. We obtain the condition formula $v \equiv_4 1$ for $take(1)$ based on $\mathcal{P}_{take(1)}$ and $\mathcal{N}_{take(1)}$. Similarly, the condition formulas for $take(2)$ and $take(3)$ are $v = 2$ and $v = 3$, respectively. Now, we have three rules: $(v \equiv_4 1, take(1))$, $(v = 2, take(2))$ and $(v = 3, take(3))$. By the verification of the winning and complete properties of the above three rules, we get that the first rule is both winning and complete and hence $take(1)$ is added into $\mathcal{A}^f$. However, the latter two rules are winning but incomplete. The verification process for completeness property returns two states 6 and 7 for $(v = 2, take(2))$ and $(v = 3, take(3))$, respectively. So the two sets $S^c_{take(2)} = \{6\}$ and $S^c_{take(3)} = \{7\}$. The set $\mathcal{A}^u$ contains the two actions $take(2)$ and $take(3)$. After combing the above three rules into a strategy δ, we check if δ is exhaustive. The exhaustiveness check returns a state 6, meaning that δ has not an applicable action for the winning state 6 s.t. its successor state is losing.

In the second iteration, the algorithm continues with the newly generated winning states $\mathcal{P}'' = \{6,7\}$. The existing actions $take(2)$ and $take(3)$ are applicable in states 6 and 7, respectively and leads them to the losing state 4. So no new actions are enumerated and $\mathcal{A}' = \emptyset$. We add the states 6 into $\mathcal{P}_{take(2)}$ and $\mathcal{N}_{take(3)}$ and 7 into $\mathcal{P}_{take(3)}$ and $\mathcal{N}_{take(2)}$. Based on the new sets of positive states and of negative states, we obtain the condition formulas $v \equiv_4 2$ and $v \equiv_4 3$ for $take(2)$ and $take(3)$, respectively. Both the two rules $(v \equiv_4 2, take(2))$ and

$(v \equiv_4 3, take(3))$ are winning and complete. The above two rules together with $(v \equiv_4 1, take(1))$ form a new strategy, which passes the exhaustiveness check. The algorithm terminates with an exhaustive winning strategy. □

5.2. Correctness verification

In this subsection, we present the constraints used to verify the winning, completeness and redundancy properties of rules, and the exhaustiveness properties of strategies.

Definition 11. Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG and ϕ a winning formula of Π.

- The constraint for the winning rule (ψ, a) is $(\mathcal{C} \wedge \phi \wedge \psi) \rightarrow \{\texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]\}$.
- The constraint for the complete rule (ψ, a) is $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']] \rightarrow \psi$.
- The constraint for the redundant rule (ψ_i, a_i) w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is $\psi_i \rightarrow \bigvee_{i \neq j} \psi_j$.
- The constraint for the exhaustive strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is $(\mathcal{C} \wedge \phi) \rightarrow (\bigvee_{i=1}^{n} \psi_i)$.

Theorem 5. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. Then the following holds:*

- *A rule (ψ, a) is winning iff the constraint for the winning rule illustrated in Definition 11 is valid.*
- *A rule (ψ, a) is complete iff the constraint for the complete rule illustrated in Definition 11 is valid.*
- *A rule (ψ_i, a_i) is redundant w.r.t. a strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ iff the constraint for the redundant rule illustrated in Definition 11 is valid.*
- *A strategy $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is exhaustive iff the constraint for the redundant rule illustrated in Definition 11 is valid.*

Proof. • The winning property of rules: ($\Rightarrow$): Assume that (ψ, a) is a winning rule. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the definition of winning rules (cf. Definition 4), a is applicable in s and $\tau(s, a)$ is a losing state. It follows that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$. Hence, the constraint for the winning rule is valid.

($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the winning rule is valid. Let s be a state satisfying $\mathcal{C} \wedge \phi \wedge \psi$. By the constraint for the winning rule, we get that $s \models \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$. It follows that a is applicable in s and $\tau(s, a)$ is a losing state. Hence, (ψ, a) is a winning rule.

• The completeness property of rules: ($\Rightarrow$): Assume that (ψ, a) is a complete rule. Let s be a state satisfying $\exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. It follows that $\tau(s, a)$ is a losing state. By the definition of complete rules (cf. Definition 4), $s \models \psi$. Hence, the constraint for the complete rule is valid.

($\Leftarrow$): Assume that (ψ, a) is a rule s.t. the constraint for the complete rule is valid. Let s be a state s.t. $\tau(s, a)$ is a losing state. It follows that $s \models \exists \mathcal{X}'[\mathcal{T}(a) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$. By the constraint for the complete rule, we get that $s \models \psi$. Hence, (ψ, a) is a complete rule.

• The redundancy property of rules: ($\Rightarrow$): Assume that (ψ_i, a_i) is a redundant rule of $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$. Let s be a state satisfying ψ_i. By the definition of redundant rules (cf. Definition 4), there is a formula ψ_j s.t. $i \neq j$ and $s \models \psi_i$. Hence, the constraint for the redundant rule is valid.

($\Leftarrow$): Assume that (ψ_i, a_i) is a rule s.t. the constraint for the redundant rule is valid. Let s be a state satisfying ψ_i. By the constraint for the redundant rule, $s \models \bigvee_{i \neq j} \psi_j$. It follows that there is a formula ψ_j s.t. $i \neq j$ and $s \models \psi_i$. Hence, (ψ_i, a_i) is is a redundant rule of $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$.

• The exhaustiveness property of strategies: ($\Rightarrow$): Assume that $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is an exhaustive strategy. Let s be a state satisfying $\mathcal{C} \wedge \phi$. By the defini-

tion of exhaustive strategies (cf. Definition 4), there is a condition formula ψ_i s.t. $s \models \psi_i$. It follows that $s \models \bigvee_{i=1}^{n} \psi_i$. Hence, the constraint for the exhaustive strategy is valid.

($\Leftarrow$): Assume that $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is a strategy s.t. the constraint for the exhaustive rule is valid. Let s be a legal winning state. It follows that $s \models \mathcal{C} \wedge \phi$. By the constraint for the constraint for the exhaustive strategy, $s \models \bigvee_{i=1}^{n} \psi_i$. It follows that there is a condition formula ψ_i s.t. $s \models \psi_i$. Hence, $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ is exhaustive.

□

Finally, we obtain the soundness theorem of Algorithm 3.

Corollary 3. *Let $\Pi = \langle \mathcal{X}, \mathcal{A}, \mathcal{C}, \mathcal{E} \rangle$ be an ICG. The set of rules returned by Algorithm 3 is an exhaustive and winning strategy of Π.*

Proof sketch: Let δ be a set of rules $\{(\psi_1, a_1), \cdots, (\psi_n, a_n)\}$ returned by Algorithm 3.

We first prove that δ is winning. Line 14 checks whether a rule (ψ_a, a) satisfies the winning property. If $S_a^w = \emptyset$, then the constraint for the winning rule is valid. By Theorem 5, (ψ_a, a) is winning. If (ψ_a, a) is complete, then Line 21 insert it into δ^c; otherwise, Line 24 inserts it into δ^w. Thus all rules of $\delta^c \cup \delta^w$ are winning. By the definition of winning strategies (cf. Definition 4), $\delta^c \cup \delta^w$ is winning.

Next, we prove the exhaustiveness of δ. Line 25 performs the exhaustiveness check of δ. If $S^e = \emptyset$, then the constraint for the exhaustive strategy is valid. By Theorem 5, δ is exhaustive. In addition, the simplification process only removes redundant rules (ψ', a') from $\delta^c \cup \delta^w$. By Theorem 5, $\psi' \rightarrow (\bigvee_{i=1}^{n} \psi_i)$. So $(\bigvee_{i=1}^{n} \psi_i) \equiv (\psi' \vee \bigvee_{i=1}^{n} \psi_i)$. We get that removing redundant rules does not affect the exhaustiveness of δ. □

6. The misère rule

In this section, we apply our approach to learning winning formulas and strategies under the misère rule. The definitions of sets of winning states and losing states are the same as Definition 1 except that all ending states are winning states. In addition, we preclude executing an action in winning but ending states. It is easy to carry forward our approach to computing winning strategies via the following minor modifications. (1) In the step of synthesizing winning formulas, classifying legal states into winning and losing states is based on the new definition. (2) The first and second constraints for winning formulas (cf. Definition 10) are replaced by $\mathcal{C} \wedge \mathcal{E} \rightarrow \phi$ and $(\mathcal{C} \wedge \neg\mathcal{E} \wedge \phi) \rightarrow \exists \mathcal{X}'[\mathcal{T}(\mathcal{A}) \wedge \neg\phi[\mathcal{X}/\mathcal{X}']]$, respectively. (3) The constraint for the winning rule (ψ, a) (cf. Definition 7) is replaced by $\mathcal{C} \wedge \neg\mathcal{E} \wedge \psi \rightarrow \texttt{pre}(a) \wedge \forall \mathcal{X}'[\mathcal{T}(a) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$.

7. Experimental evaluation

This section first presents implementation of our proposed algorithms and the benchmarks we establish in the experiments, and then analyzes the experimental results.

7.1. Implementation

We implemented the synthesis approach for winning formulas and strategies, illustrated in Algorithms 1 and 3 in Python 3.7. The two variants, based on different candidate formula generation methods, are referred to as SynEnum (enumeration-based) and SynMS (exact binary classification-based). Our tool takes as input a PDDL-like file that formalizes an ICG and outputs the winning formula and strategy for this ICG. We employ Z3 [18] as the SMT solver to verify the correctness of the winning formula and strategy as well as to determine if a state is winning. We adopt NuWLS [33] as the partial MaxSAT solver in order to choose the small set of generated arithmetic atoms. NuWLS is an incomplete partial MaxSAT solver and may return a sub-optimal solution.

This however has no influence on the soundness of the chosen set as the solution satisfies the hard constraint. By Theorem 3, we can always construct a formula from the chosen set that distinguishes the set of positive states from that of negative states. The time-out bounds for Z3 and NuWLS are 10 and 3 seconds, respectively. The experiments were conducted on a PC with an Intel Core i7-7700 3.60 GHz CPU and 16GB RAM running under Windows 10, with a runtime cutoff of $1,200$ seconds. Source codes are provided in this link[2].

7.2. Benchmarks

We evaluated the performance of our approach SynEnum and SynMS on a large number of ICG domains that consist of the following 5 categories: Subtraction [34], Nim [5], Wythoff [6], Welter [35] and Chomp [7]. Each category contains multiple classes of ICGs, including some extensions or variants of the original game. The PDDL file of each test case are given in this link[3]. Since the winning formula and strategy are not LIA-definable or unknown in some classes of ICGs, we impose certain limitations on these classes in order to guarantee that they have LIA-definable solution. For example, the winning formula of 3-rowed Chomp game is unknown. We restrict the maximum number of cookies on the 3rd row to be a fixed integer and obtain a restricted version of 3-rowed Chomp which has an LIA-definable solution. In summary, the benchmark consists of a total of 32 classes with $3,720$ test cases, including the 46 test cases in [21].

7.2.1. Subtraction Games

This family consists of single-heap games in which each move changes the position only by decreasing one numeric state variable. Its rule structure is the simplest among our benchmarks, so it provides a natural starting point for evaluating whether the synthesis methods can discover basic arithmetic regularities in one-dimensional state spaces. We include Take Away [11], Subtraction

[2]https://github.com/caro2357/SynMS
[3]https://github.com/caro2357/Domain

[34], and S, D-Mark [13]. Taken together, these subtraction-style benchmarks contribute 479 experimental instances under each play rule.

Take Away. There is a pile of v chips. A move consists of taking any number from 1 to m chips. This class provides the simplest single-heap benchmark, because the state is determined by only one variable and the move set forms a continuous interval. Each test case differs in the parameter m, which is at most 30, and there are 30 test cases for each play rule.

Subtraction. Let S be a finite set of positive integers. A player may take k chips from the pile only when $k \in S$. Compared with Take Away, this class replaces the continuous interval of legal moves with a discrete subtraction set, so the strategic structure depends more directly on the choice of S. We consider sets S of size at most 4, and the resulting benchmark contains 385 test cases.

S, D-Mark. Given two finite sets of integers S and D, a move may either remove k chips with $k \in S$ or replace v by $\frac{v}{d}$ when $d \in D$ divides v. This variant is structurally richer than the previous two classes, because subtraction moves and division-style moves coexist in the same single-heap rule system. For the benchmark, S is instantiated as $[1, 2, \cdots, t-1]$ with $2 \leq t \leq 9$, D contains one divisor d with $2 \leq d \leq 9$, and this yields 64 test cases.

7.2.2. Nim Games

This category moves from one heap to several interacting heaps, so the strategic structure may depend on order, parity, locality, redistribution, or extra control information rather than on a single counter alone. Accordingly, it serves as our main benchmark family for testing how well the synthesis methods handle higher-dimensional states and relations among multiple variables. The family contains Nim [5], Monotonic Nim [10], Empty and Transfer [8], Empty All But One [8], Empty and Redistribute [8], and Entropy Reduction Game [8]. It also includes Min Game [36], l-Slow Nim [10], Sharing Nim [37], Small Nim [38], Greedy Nim [39], Odd-or-Even Nim [40], Modular Nim [41], Pointed Nim [39], End Nim [42], Loop End Nim [42], Staircase Nim [43], and Circular Nim [44]. In total, the Nim family contributes 302 experimental instances under each play

rule. **Nim**. There are n piles of chips where v_i denotes the number of chips of the i-th pile. Each move removes a positive number of chips from exactly one pile. This class is the canonical multi-heap benchmark, so it serves as the baseline for evaluating whether the synthesis methods can handle higher-dimensional states. Because the winning formula of Nim requires exclusive-or and is therefore not LIA-definable, we restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and also include one unrestricted version, obtaining 7 test cases.

Monotonic Nim. A player may remove chips from one pile only if the resulting sequence remains non-decreasing, that is, $v_i \geq v_{i+1}$. The distinguishing feature of this class is that each move must preserve a global order relation among heaps rather than merely reduce one component. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Empty and Transfer. A move empties one pile, selects another pile, and transfers some but not all of its chips into the emptied pile. Unlike standard Nim, this class emphasizes redistribution between heaps and therefore captures interactions that cannot be reduced to independent single-heap decreases. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

Empty All But One. A move empties $n - 1$ piles and redistributes the chips of the remaining pile among all piles so that each pile remains nonempty. The key feature here is whole-position redistribution, because a legal action rewrites the entire heap configuration in one step rather than changing a single heap locally. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and obtain 6 test cases in total.

Empty and Redistribute. One pile is first discarded completely, and the chips in the remaining piles are then redistributed among all piles so that each pile contains at least one chip. This class is more global than ordinary Nim, because discarding one heap and repartitioning the rest changes the full state structure at once. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and obtain 6 test cases in total.

Entropy Reduction Game. A move transfers chips from one pile to another in such a way that the heaps become more nearly equal in size. Its characteristic feature is that a legal move is guided by reducing dispersion across heaps instead of simply decreasing one heap independently. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Min Game. There are two piles of chips v_1 and v_2, and a move removes k chips from the larger pile where k is bounded by the smaller pile. The strategy of this class depends directly on comparing the two heaps, because the legal amount that may be removed is determined by the current minimum. We restrict the maximum of v_2 to be at most 5 and also include the unrestricted version, giving 6 test cases in total.

l-Slow Nim. The number of chips removed in one move is restricted to the interval $[1, l]$. This class may be viewed as Nim with an explicit speed limit, so it separates the effect of bounded local decreases from the unrestricted baseline. We restrict the number of piles to 3, bound the third pile by $m \leq 5$, let l range up to 30, and obtain 180 test cases.

Sharing Nim. In one turn, a player takes some chips from one pile and adds them to another pile, provided that after the move the destination pile does not exceed the source pile. This class emphasizes redistribution under balance constraints, and therefore tests interactions among heaps beyond pure deletion. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, yielding 7 test cases.

Small Nim. A player may remove chips only from a smallest heap, or from any heap tied for the smallest size. Its main feature is a strong local-minimum restriction, so legal play is determined by the current order among heaps. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Greedy Nim. A player may remove chips only from a largest heap, or from any heap tied for the largest size. As the dual of Small Nim, this class imposes a maximal-heap preference and highlights how local extremal choices shape the

strategy. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Odd-or-Even Nim. At the beginning, the first player specifies whether the opponent must take an odd or an even number of chips, and each later move updates this parity requirement for the next round. This class differs from standard Nim by adding a control variable, so legal actions depend on both heap sizes and the current parity mode. We restrict the number of piles to at most 2, bound the second pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

Modular Nim. A move removes k chips from one pile subject to a modular arithmetic condition $k \not\equiv_n x$. This class introduces arithmetic constraints directly into the move rule, so strategically relevant patterns can depend on residue classes instead of only on magnitudes. We restrict the number of piles to at most 3 and enumerate all residues $0 \leq x < n$, which gives 5 test cases.

Pointed Nim. A pointer indicates from which heap the next move must be made, and after removing chips the player must redirect the pointer to a different nonempty heap. The crucial feature of this class is the additional control component, because the next legal move depends on the current pointer as well as the heap sizes. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, obtaining 7 test cases.

End Nim. The piles are ordered, and a player may remove chips only from the front-end or back-end nonempty pile. This class adds a positional restriction to Nim, because play is limited to boundary heaps rather than any heap in the sequence. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Loop End Nim. This variant keeps the end-move restriction of End Nim, but the game ends when only one pile remains nonempty. Its distinctive feature lies in the altered terminal condition, which changes the strategic role of reducing the position to a single surviving heap. We restrict the number of piles to at most 3, bound the third pile by $m \leq 5$, and include the unrestricted version,

obtaining 7 test cases.

Staircase Nim. A staircase of n steps contains coins on some steps, and a move transfers any positive number of coins from step v_i to the next lower step v_{i-1}. This class differs from ordinary Nim because tokens move along adjacent positions instead of being removed directly from an isolated heap. We restrict the number of steps to at most 3, bound the third step by $m \leq 5$, and include the unrestricted version, giving 7 test cases.

Circular Nim. There are n piles arranged in a circle, and one move chooses m consecutive piles and removes at least one chip from one or more of them. The defining feature of this class is circular adjacency, which makes the global layout of heaps part of the legal move relation. We consider the cases with $1 \leq n \leq 5$ and $0 \leq m \leq n$, which gives 15 test cases.

7.2.3. Welter Games

Unlike the heap-based families above, this category represents a position by token locations on a line rather than by pile sizes. Its characteristic challenge comes from ordered movement and non-overlap constraints, which makes it a compact benchmark for testing whether the synthesis methods adapt smoothly to movement-based state spaces. We consider Welter [35], l-Welter [45], and Max-Welter [46]. All together, these Welter games contribute 7 experimental instances under each play rule. **Welter**. There are n coins placed on distinct squares of a semi-infinite linear board, and v_i denotes the position of the i-th coin. A legal move sends one coin to a lower unoccupied square. This class differs from the previous heap-based games because the state is defined by ordered token positions and non-overlap constraints rather than heap sizes. We restrict the number of coins to be 2, so there is 1 test case in this game.

l-Welter. Each move may send a coin to a lower unoccupied square, but the distance of a single move is bounded by l. This restriction makes the game more local than standard Welter, because long token moves are explicitly forbidden. We restrict the number of coins to 2 and let l range up to 5, which gives 5 test cases.

Max-Welter. Only the coin occupying the largest square may be moved to a lower unoccupied square. The main strategic constraint is therefore determined by the maximal token position, which makes the order of coins central to the game. We restrict the number of coins to be 2, so there is 1 test case in this game.

7.2.4. Wythoff-Type Games

This family centers on two-heap games in which ordinary Nim-style moves coexist with synchronized changes to both heaps, so the geometry of the state space becomes central to the winning pattern. Because it combines rich arithmetic structure with a very large collection of parameterized variants, it is also the largest and most demanding benchmark category in our experiments. We include Wythoff [6], l-Wythoff [47], Monotonic l-Diet Wythoff [10], Wythoff with subtraction set S [48], and Wythoff with a function f [9]. In addition, the category contains the 1st (l_1, l_2)-restricted Wythoff [49], the 2nd (l_1, l_2)-restricted Wythoff [50], the 3rd (l_1, l_2)-restricted Wythoff [51], and the 4th (l_1, l_2)-restricted Wythoff [52]. Overall, this Wythoff-type family contributes $2,924$ experimental instances under each play rule. **Wythoff**. There are two piles of chips, and each move may either remove chips from one pile only or remove the same number of chips from both piles. The coexistence of Nim moves and synchronized diagonal moves makes this class structurally richer than standard Nim. We restrict the second pile by $m \leq 5$ and include one unrestricted version, which gives 21 test cases in total.

l-Wythoff. A Wythoff move may remove k_1 chips from the first pile and k_2 chips from the second pile provided that $|k_1 - k_2| < l$. This class relaxes the strict equality of diagonal moves and therefore interpolates between standard Wythoff play and more flexible synchronized reductions. We bound both l and the second pile by 5 and include the unrestricted version of v_2, obtaining 140 test cases.

Monotonic l-Diet Wythoff. This variant requires $v_1 \geq v_2$ and bounds the number of removed chips by l. It combines restricted diagonal play with

an additional monotonicity condition, so both synchronization and heap order become essential. We bound both l and the second pile by 5 and include the unrestricted version of v_2, giving 105 test cases.

Wythoff with subtraction set S. Let S be a finite set of positive integers, and in a Wythoff move each player removes $k \in S$ chips from both piles. This class combines the discrete move-set flavor of subtraction games with the synchronized two-pile mechanism of Wythoff games. We consider sets S of size at most 2, let each element of S range from 1 to 5, bound the second pile by $m \leq 5$, and include the unrestricted version, obtaining 315 test cases.

Wythoff with a function f. In a Wythoff move, the removed pair (k_1, k_2) must satisfy $0 < k_1 \leq k_2 < f(k_1)$. The key feature here is that the allowed synchronized move is controlled by a parameterized function rather than by fixed constants or a finite set. We restrict the second pile by $m \leq 5$, use functions of the form $f(k) = k + i$ with $0 \leq i \leq 5$, include the unrestricted version of v_2, and obtain 168 test cases.

1st (l_1, l_2)**-restricted Wythoff**. After a diagonal move, the reached state (v_1, v_2) must satisfy $\min(v_1, v_2) \geq l_1$ and $\max(v_1, v_2) \geq l_2$. This restriction excludes a forbidden corner of the state space, so diagonal play is constrained by the geometry of the target position. We bound l_1, l_2, and the second pile by 5, include the unrestricted version of v_2, and obtain 315 test cases.

2nd (l_1, l_2)**-restricted Wythoff**. A diagonal move is allowed when either $\min(k_1, k_2) < l_1$ or $|k_1 - k_2| < l_2$. This class controls synchronized play through a disjunction of minimum-based and difference-based constraints, yielding a more flexible restricted diagonal rule. We bound l_1, l_2, and the second pile by 5, include the unrestricted version of v_2, and obtain 600 test cases.

3rd (l_1, l_2)**-restricted Wythoff**. In a diagonal move, the difference between the removed amounts must satisfy $0 \leq |k_2 - k_1| < (l_1 - 1)\lambda + l_2$ where $\lambda = \min(k_1, k_2)$. The notable feature of this class is that the admissible difference is itself state-dependent through λ, which yields a more expressive restriction on diagonal play. We bound l_1, l_2, and the second pile by 5, include the unrestricted version of v_2, and obtain 700 test cases.

4th (l_1, l_2)-restricted Wythoff. In a diagonal move, the removed amounts must satisfy the linear proportionality condition $k_1 \cdot l_1 = k_2 \cdot l_2$. This class characterizes synchronized play through a proportional arithmetic relation, which yields a different geometric pattern from the previous restricted variants. We bound l_1, l_2, and the second pile by 5, include the unrestricted version of v_2, and obtain 560 test cases.

7.2.5. Chomp Games

This final family replaces heaps or token positions with monotone board shapes that shrink by deleting entire regions. Because a single move usually changes many cells at once, these games provide representative shape-based benchmarks whose strategies depend on global geometry rather than on local arithmetic alone. We include Chomp [7] and L-shaped Chomp [7]. This category contributes 8 experimental instances under each play rule. **Chomp**. Cookies are laid out on rows, v_i denotes the number of cookies on row i, and a move eats one remaining cookie together with all cookies to its right and above it. This class is markedly different from the previous benchmarks because the state is a monotone board shape, and one action usually changes many cells at the same time. Since the winning strategy of unrestricted Chomp is unknown, we restrict the number of rows to at most 3, bound the third row by $m \leq 5$, and obtain 7 test cases.

L-shaped Chomp. The board has a fixed L-shape, and the variables v_1 and v_2 denote the lengths of the vertical and horizontal arms. This restricted form keeps the region-removal mechanism of Chomp while offering a much smaller and cleaner state space for evaluation. There is 1 test case in this class.

7.3. Experimental Results

We compared the performance of the SynEnum and SynMS under the normal and misère rules, respectively, and provided detailed experimental data tables. In each table, the columns "Solved" denotes the number of test cases that solved by each approach, and the figure in parentheses denotes the number

of test cases that solved by the corresponding approach but not by the other approach. The columns "Time" and "Size" denote the average time and size of solved test cases for each approach, and the figure in parentheses denotes the average time and size of the test cases solved by both approaches, respectively. The optimal data of each group are in bold.

7.3.1. Experimental Results under normal rule

Comparison on Sub game. We summarize the results of two approaches about synthesizing winning formulas in the Subtraction game, as shown in Table 4. Out of 479 test cases, SynEnum successfully solved 124 test cases, while SynMS solved 363 test cases, achieving a synthesis success rate of 75.7%, which is approximately 2.93× higher than SynEnum's. Notably, we found that for each category of game, SynMS is able to solve all the test cases that SynEnum can solved. For the commonly solved 124 test cases, we can see that the running time of SynEnum is 159.13 seconds, compared to just 6.99 seconds of SynMS, resulting in a significant reduction.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	11.2(11.20)	**8.87**(**8.87**)	**6.97**(**6.97**)	8.93(8.93)
Subtraction	385	86(0)	**315**(229)	201.89(201.89)	**63.73**(**6.68**)	**10.92**(**10.92**)	64.93(25.71)
T,d-I-Mark-Game	64	8(0)	**18**(10)	254.1(254.10)	**100.06**(**3.28**)	**18.88**(**18.88**)	76.22(71.75)
Total	479	124(0)	**363**(239)	159.13(159.13)	**61.0**(**6.99**)	**10.48**(**10.48**)	60.86(24.62)

Table 4: Results of synthesizing winning formulas for Sub games under the normal rule

For a deeper analysis of runtime behavior, we show the number of solved test cases after a certain amount of time in Figure 2(a). In only 10 seconds, SynMS can solve approximately 284 test cases. In comparison, SynEnum only completes 124 test cases within the timeout $1,200$ seconds. As for the size, SynMS generates formulas larger than SynEnum, with an average size 24.62 on commonly solved test cases, while SynEnum has an average size 10.48. In theory, Enum is guaranteed to find the smallest-size formula via exhaust enumeration[53].

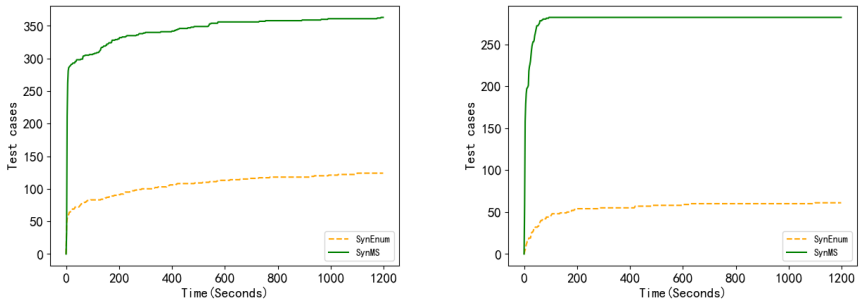
Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	14(0)	**30**(16)	58.54(58.54)	**16.86**(**4.51**)	**91.43**(**91.43**)	236.13(106.07)
Subtraction	385	45(1)	**247**(203)	121.33(99.11)	**10.71**(**2.33**)	**46.58**(**46.14**)	306.86(117.84)
T,d-I-Mark-Game	64	2(0)	**5**(3)	206.96(206.96)	**28.50**(**18.64**)	**64.0**(**64.00**)	130.4(79.00)
Total	479	61(1)	**282**(222)	109.73(93.24)	**11.68**(**3.38**)	**57.44**(**57.30**)	296.21(113.80)

Table 5: Results of synthesizing winning strategies for Sub games under the normal rule

SynMS constructs the winning formula in disjunctive normal form based on the selected atoms. In addition, any formula is exponentially smaller than its equivalent one in disjunctive normal form. It is noticeable that SynMS achieves a better performance in computing winning formulas compared to SynEnum although it sacrifices solution size.

Table 5 summarizes the results about synthesizing winning strategies. SynEnum successfully solved 61 test cases, with a success rate of 49.1% and an average running time of 109.73 seconds, while SynMS successfully solved 282 test cases, with a success rate of 77.6% and an average running time of 11.68 seconds. Whether it is the running time for commonly solved test cases or the running time for all test cases, SynMS outperforms SynEnum. This is also evident in Figure 2(b), where SynMS is able to solve 278 test cases in 60 seconds, while SynEnum can only solve 36 test cases. It is worth noting that SynEnum successfully solves 1 test case that SynMS cannot. The winning formulas of the test case is an arithmetic literal (that is, an arithmetic atom or its negation) of large size. SynEnum is more effective than SynMS at solving the test case since it can enumerate formulas up to size 25. SynMS takes some time to choose arithmetic atoms and combine the chosen atoms into an arithmetic formula and hence generates arithmetic atoms of at most size 7. For the commonly solved 60 test cases, the comparison of formula size is consistent with the performance observed when synthesizing winning formulas.

Comparison on Nim game. Table 6 presents the results of two approaches about synthesizing winning formulas in the Nim game. The findings



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 2: Comparison of the running time of synthesizing winning formula and strategy for Sub game under normal rule

reveal that the number of test cases successfully solved by SynEnum and SynMS is comparable. Specifically, SynEnum successfully solved 66 test cases with an average running time of 138.21 seconds, while SynMS solved 73 test cases, but with a faster average running time of 51.82 seconds. The most notable difference between the two approaches is observed in the l-Slow Nim game. Furthermore, we found that there are 29 test cases that SynEnum can solve, but SynMS cannot, a pattern similar to what was observed in the Subtraction game. For the 37 test cases that are commonly solved, SynEnum's average running time is 62.96 seconds, while SynMS's average running time is 23.94 seconds, making SynMS approximately 2.62× faster than SynEnum. However, we found that in certain categories of games, SynEnum performs faster than SynMS on the commonly solved test cases. For example, in the Monotonic Nim game, SynEnum's average running time is 2.29 seconds, while SynMS takes 18.43 seconds. This is due to the simplicity of these test cases. Although the average running time of SynMS is generally slightly longer than that of SynEnum for these test cases, the time difference between the two approaches typically stays within 50 seconds. Further analysis in Figure 3(a) shows that both approaches are able to solve 75% of the test cases they can solve within 100 seconds. Regarding the size of the formulas, similar to the case in Subtraction game, the formulas generated by

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	1(0)	**2**(1)	**0.51**(**0.51**)	80.84(3.21)	**4.00**(**4.00**)	59.5(6.00)
Monotonic Nim	7	**7**(0)	**7**(0)	**2.29**(**2.29**)	18.43(18.43)	**8.29**(**8.29**)	12.57(12.57)
Empty and Transfer	7	1(0)	**2**(1)	100.92(100.92)	**36.41**(**15.91**)	**12.0**(**12.00**)	26.5(43.00)
Empty All But One	6	2(0)	**3**(1)	269.48(269.48)	**26.17**(**21.04**)	**17.50**(**17.50**)	73.0(45.00)
Empty and Redistribute	6	0(0)	**2**(2)	-(-)	107.91(-)	-(-)	97.50(-)
Entropy Reduction Game	7	0(0)	**1**(1)	-(-)	40.33(-)	-(-)	68.00(-)
Min Game	6	1(0)	**2**(1)	**3.78**(**3.78**)	5.73(7.44)	**10.00**(**10.00**)	67.5(16.00)
l-Slow Nim	180	**35**(27)	14(6)	204.32(56.46)	**38.32**(**42.42**)	**11.60**(**10.25**)	103.21(91.25)
Sharing Nim	7	1(0)	**6**(5)	**0.07**(**0.07**)	83.27(0.73)	**1.00**(**1.00**)	34.83(**1.00**)
Small Nim	7	2(0)	**6**(4)	597.29(597.29)	**24.07**(**6.93**)	**16.50**(**16.50**)	49.33(62.00)
Greedy Nim	7	1(0)	**6**(5)	**0.55**(**0.55**)	159.66(2.98)	**4.00**(**4.00**)	285.17(6.00)
Odd-or-Even Nim	7	0(0)	**6**(6)	-(-)	47.21(-)	-(-)	415.50(-)
Modular Nim	5	**1**(0)	**1**(0)	**1.49**(**1.49**)	2.36(2.36)	**8.00**(**8.00**)	25.0(25.00)
Odd-or-Even Nim	7	0(0)	**6**(6)	-(-)	47.21(-)	-(-)	415.50(-)
Pointed Nim	7	0(0)	**1**(1)	-(-)	221.87(-)	-(-)	44.00(-)
End Nim	7	1(0)	**2**(1)	**5.54**(5.54)	20.48(**1.61**)	**13.00**(**13.00**)	157.5(**13.00**)
Loop End Nim	7	1(0)	**2**(1)	**5.42**(5.42)	32.90(7.04)	**13.00**(**13.00**)	76.5(**13.00**)
Staircase Nim	7	**7**(0)	**7**(0)	**1.09**(**1.09**)	42.98(42.98)	**4.00**(**4.00**)	9.0(9.00)
Circular Nim	15	**5**(2)	3(0)	19.05(**0.66**)	**6.52**(6.52)	**7.40**(**4.67**)	12.0(12.00)
Total	302	66(29)	**73**(36)	138.21(62.95)	**51.82**(**23.94**)	**10.03**(**8.51**)	105.03(33.92)

Table 6: Results of synthesizing winning formulas for Nim games under normal rule

SynMS are still larger than those generated by SynEnum.

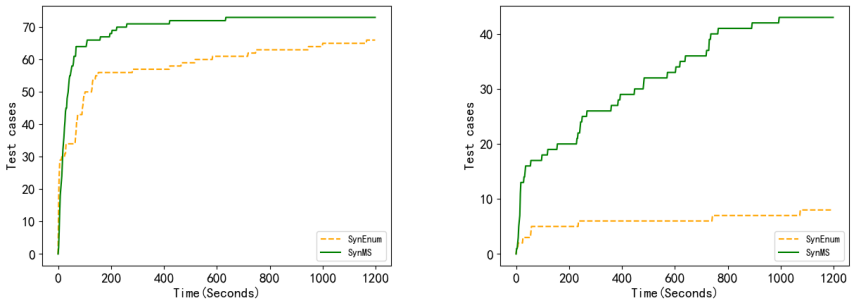
Table 7 presents the results about synthesizing winning strategies. We can observe that the comparison between SynEnum and SynMS, in terms of the number of solved test cases and average running time, is similar, just as it is in the synthesis of winning formulas. The only difference is that, among the 8 commonly solved test cases, SynMS produces a smaller strategy size in 3 of them compared to SynEnum. The main reason for this phenomenon is that SynEnum is able to generate a conditional formula ψ_1 containing larger arithmetic atomic formulas for a specific action, which SynMS cannot generate. However, SynMS can generate conditional formulas for other actions thereby

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	0(0)	**1**(1)	-(-)	13.68(-)	-(-)	44.00(-)
Monotonic Nim	7	0(0)	**1**(1)	-(-)	15.87(-)	-(-)	22.00(-)
Empty and Transfer	7	0(0)	**1**(1)	-(-)	15.73(-)	-(-)	284.00(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	**7.77**(**7.77**)	16.71(28.14)	**84.00**(**84.00**)	294.5(154.00)
l-Slow Nim	180	1(0)	**7**(6)	**234.69**(234.69)	384.28(**34.92**)	**108.00**(**108.00**)	358.57(135.00)
Sharing Nim	7	1(0)	**5**(4)	**1.11**(1.11)	79.92(**0.89**)	**1.00**(**1.00**)	185.00(**1.00**)
Small Nim	7	0(0)	**3**(3)	-(-)	353.01(-)	-(-)	463.33(-)
Greedy Nim	7	0(0)	**3**(3)	-(-)	327.74(-)	-(-)	526.00(-)
Odd-or-Even Nim	7	0(0)	**5**(5)	-(-)	306.66(-)	-(-)	1356.60(-)
Modular Nim	5	0(0)	**1**(1)	-(-)	7.34(-)	-(-)	246.00(-)
Pointed Nim	7	0(0)	**1**(1)	-(-)	602.37(-)	-(-)	365.00(-)
End Nim	7	1(0)	**2**(1)	**56.36**(56.36)	501.19(**10.22**)	**149.00**(149.00)	608.00(**74.00**)
Loop End Nim	7	1(0)	**2**(1)	**55.73**(55.73)	450.56(**9.81**)	**149.00**(149.00)	451.00(**74.00**)
Staircase Nim	7	2(0)	**7**(5)	**383.21**(383.21)	479.87(**119.13**)	**155.50**(**155.50**)	399.71(230.00)
Circular Nim	15	1(0)	**2**(1)	1072.6(1072.60)	**25.09**(**32.22**)	126.0(126.00)	**29.00**(**14.00**)
Total	302	8(0)	**43**(35)	**274.33**(274.33)	294.57(**44.31**)	**116.00**(116.00)	458.37(**114.00**)

Table 7: Results of synthesizing winning strategies for Nim games under normal rule

indirectly covering all winning states that satisfy the conditional formula ψ_1. Specifically, suppose SynEnum can synthesize the conditional formula ψ_1 for action a_1, where ψ_1 contains larger arithmetic atomic formulas. However, SynMS can synthesize the conditional formula ψ_2 for action a_2, and for any state s, if $s \models \texttt{pre}(a_1) \wedge \forall \mathcal{X}'[\mathcal{T}(a_1) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$ holds, then it must also be true that $s \models \texttt{pre}(a_2) \wedge \forall \mathcal{X}'[\mathcal{T}(a_2) \rightarrow \neg\phi[\mathcal{X}/\mathcal{X}']]$ holds. Since SynEnum can also synthesize the corresponding conditional formulas for action a_2, SynMS typically ends up with a smaller strategy size than SynEnum.

Comparison on Welter game. We summarize the results of two approaches about synthesizing winning formulas in the Welter game, as shown in Table 8. SynEnum successfully solved 3 test cases, while SynMS solved 4 test



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 3: Comparison of the running time of synthesizing winning formula and strategy for Nim game under normal rule

cases, including 2 commonly solved test cases and 2 test cases that SynEnum can not solve but SynMS can. In terms of average running time on the commonly sloved test cases, the difference between the two approaches is very small and can almost be ignored. As shown in Figure 4(a), the trends in solving test cases

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	**1**(1)	-(-)	8.04(-)	-(-)	73.00(-)
I_welter Game	5	**3**(1)	2(0)	**3.75**(**3.71**)	6.13(6.13)	**8.67**(**8.50**)	88.5(88.50)
Max_welter Game	1	0(0)	**1**(1)	-(-)	7.83(-)	-(-)	73.00(-)
total	7	3(1)	**4**(2)	**3.75**(**3.71**)	7.03(6.13)	**8.67**(**8.50**)	80.75(88.50)

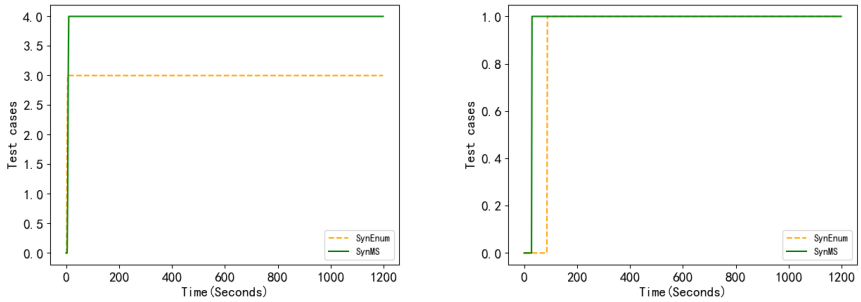
Table 8: Results of synthesizing winning formulas for Welter games under normal rule

over time are very similar for both approaches, with both stabilizing at nearly the same point. Regarding the average formula size for the commonly solved test cases, the formula size of SynEnum is still smaller than that of SynMS. In terms of strategy synthesis, both SynEnum and SynMS solved only one test case, with their performance in average running time and average strategy size being similar to that of winning formula synthesis.

Comparison on Wythoff game. Table 10 presents the results about

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
I_welter Game	5	**1**(0)	**1**(0)	87.66(87.66)	**29.22**(**29.22**)	**72.0**(**72.00**)	107.0(107.00)
Max_welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
total	7	**1**(0)	**1**(0)	87.66(87.66)	**29.22**(**29.22**)	**72.00**(**72.00**)	107.00(107.00)

Table 9: Results of synthesizing winning strategies for Welter games under normal rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 4: Comparison of the running time of synthesizing winning formula and strategy for Welter game under normal rule

synthesizing winning formulas in the Wythoff game. The results show that both SynEnum and SynMS perform excellently in the Wythoff game, successfully solving over $1,700$ test cases, with SynMS solving 582 more test cases than SynEnum. In addition, SynMS is able to solve 647 test cases that SynEnum can not solve. For the commonly solved 1640 test cases, the average running time of SynMS is 8.34 seconds, while SynEnum's is 50.47 seconds, about $6\times$ longer than SynMS. Despite the time difference, it is worth noting that for each game category, the average running time for both approaches stays within a relatively reasonable range on the commonly solved test case, under 125 seconds. As for the formula size, for each game category, SynEnum typically synthesizes smaller formulas, usually around 15. In contrast, SynMS tends to synthesize larger for-

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	10(0)	**17**(7)	124.9(124.9)	**106.53**(**5.39**)	**10.10**(**10.01**)	89.82(28.10)
l_Wythoff_Game	140	62(6)	**89**(33)	85.19(89.51)	**50.31**(**26.34**)	**10.6**(**10.84**)	127.35(67.12)
Monotonic_l_Diet_Game	105	76(17)	**78**(19)	54.46(51.28)	8.**36**(**6.87**)	**9.62**(**9.41**)	128.29(29.25)
Wythoff with subtraction set S	315	153(1)	**258**(106)	42.89(40.97)	**36.04**(**5.75**)	8.05(7.99)	70.98(26.86)
Wythoff with a function f	168	95(7)	**125**(37)	58.08(49.80)	**41.20**(**9.18**)	**8.65**(**8.70**)	97.09(55.82)
1st(l_1,l_2)restricted Wythoff	315	162(0)	**251**(89)	53.1(53.10)	**33.97**(**5.07**)	**6.84**(**6.84**)	77.96(20.09)
2nd(l_1,l_2)restricted Wythoff	600	503(1)	**575**(73)	21.8(21.36)	**13.22**(**4.92**)	**7.80**(**7.78**)	41.48(22.31)
3rd(l_1,l_2)restricted Wythoff	700	296(33)	**408**(145)	136.89(118.87)	**24.24**(**19.68**)	**10.71**(**11.12**)	119.59(72.56)
4th(l_1,l_2)restricted Wythoff	560	348(0)	**486**(138)	35.31(35.31)	**17.84**(**4.59**)	**6.9**(**6.85**)	57.45(15.89)
Total	2924	1705(65)	**2287**(647)	55.79(50.47)	**24.52**(**8.34**)	**8.34**(**8.29**)	75.84(33.82)

Table 10: Results of synthesizing winning formulas for Wythoff games under normal rule

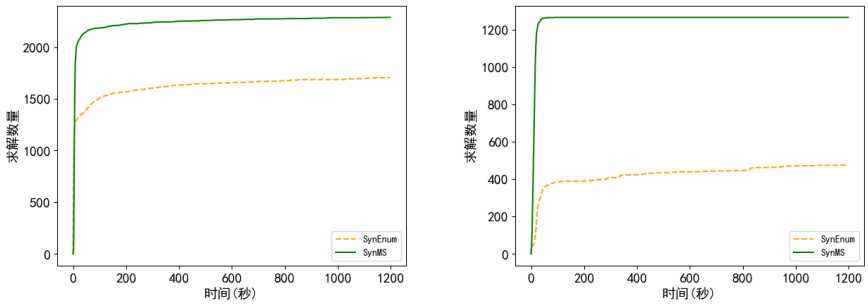
mulas, once again reflecting the trade-off between the size of the solutions and the running time during the synthesis of the winning formulas. Further analysis of Figure 5 (a) shows that both approaches can solve over 1200 test cases within only 5 seconds, with these test cases being relatively simple. The distinction between the two approaches becomes evident after 5 seconds: SynEnum's solving speed gradually slows down over time, while SynMS maintains a consistently fast solving speed.

The results about synthesizing winning strategies are shown in the Table 11 and Figures 5(b). It is evident that SynEnum significantly lags behind SynMS in terms of synthesizing winning strategies. SynEnum successfully solved only 477 test cases, with a success rate of less than 30%, while SynMS solved $1,266$ test cases and achieved a success rate of over 50%. This difference arises because the winning strategies for these test cases are complex, which are composed of a larger number of arithmetic atoms. Figure 5 (b) further shows that within 50 seconds, SynMS can solve over 1,200 test cases, whereas SynEnum can handle fewer than 400. It is worth mentioning although SynEnum's average running time for each category game is still higher than that of SynMS on the commonly solved test cases, the running time for both approaches did not

exceed 160 seconds and remained within a short time range. At the same time, whether in synthesizing winning formulas or winning strategies, the average size of SynEnum is smaller than that of SynMS.

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	2(0)	**8**(6)	27.42(27.42)	**13.36**(**11.21**)	**85.00**(**85.00**)	211.5(41.50)
1_Wythoff_Game	140	1(0)	**51**(50)	20.32(20.32)	**13.90**(**11.07**)	**82.00**(**82.00**)	349.92(**82.00**)
Monotonic_1_Diet_Game	105	24(2)	**71**(49)	63.95(69.72)	**8.75**(**4.97**)	**31.29**(**33.09**)	227.7(46.05)
Wythoff with subtraction set S	315	52(0)	**128**(76)	48.33(48.33)	**11.94**(**11.62**)	**104.08**(**104.08**)	279.27(148.73)
Wythoff with a function f	168	4(0)	**76**(72)	16.93(16.93)	**10.84**(**12.55**)	**82.00**(**82.00**)	287.34(98.75)
1st(l_1,l_2)restricted Wythoff	315	86(0)	**136**(50)	71.14(71.14)	**14.18**(**15.66**)	**117.76**(**117.76**)	277.54(203.40)
2nd(l_1,l_2)restricted Wythoff	600	36(1)	**211**(176)	**0.42**(0.06)	3.2(**0.05**)	**2.69**(**1.00**)	102.41(**1.00**)
3rd(l_1,l_2)restricted Wythoff	700	93(4)	**243**(154)	157.23(134.83)	16.60(8.67)	104.55(100.7)	420.33(125.52)
4th(l_1,l_2)restricted Wythoff	560	179(0)	**342**(163)	194.79(194.79)	11.21(11.62)	127.73(127.73)	282.42(218.22)
Total	2924	477(7)	**1266**(796)	125.4(121.65)	**11.26**(**10.63**)	**103.88**(**103.63**)	277.54(163.97)

Table 11: Results of synthesizing winning strategies for Wythoff games under normal rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 5: Comparison of the running time of synthesizing winning formula and strategy for Wythoff game under normal rule

Comparison on Chomp game. We summarize the results of two approaches about synthesizing winning formulas in the Chomp game, as shown in Table 12. Both SynEnum and SynMS are only able to solve 2 test cases.

Since the winning formulas for these test cases are relatively simple, SynEnum's average solving time is shorter than that of SynMS on the commonly solved test cases. Additionally, the average size of the winning formulas for these test cases is identical, with both having a size of 6.

Table 13 presents the results about synthesizing winning strategies. Both SynEnum and SynMS are able to solve 1 test case, and the comparison of their average running times and sizes is consistent with those observed in the synthesis of winning formulas. Figure 6 further confirms this.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**1.78**(**1.78**)	7.26(7.26)	**8.00**(**8.00**)	**8.00**(**8.00**)
L_shape Chomp Game	1	**1**(0)	**1**(0)	**1.09**(**1.09**)	9.1(9.10)	**4.00**(**4.00**)	**4.00**(**4.00**)
Total	8	**2**(0)	**2**(0)	**1.44**(**1.44**)	8.19(8.18)	**6.00**(**6.00**)	**6.00**(**6.00**)

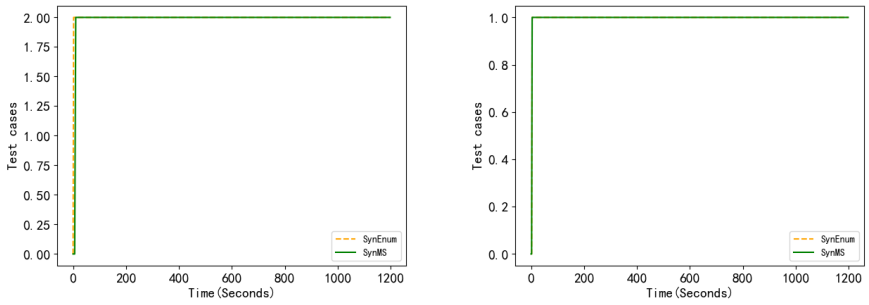
Table 12: Results of synthesizing winning formulas for Chomp games under normal rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**2.30**(**2.30**)	3.50(3.50)	**29.00**(**29.00**)	**29.00**(**29.00**)
L_shape Chomp Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	8	**1**(0)	**1**(0)	**2.30**(**2.30**)	3.50(3.50)	**29.00**(**29.00**)	**29.00**(**29.00**)

Table 13: Results of synthesizing winning strategies for Chomp games under normal rule

7.3.2. Experimental Results under misère rule

Comparison on Sub game. Table 14 presents the results about synthesizing winning formulas under the misère rule. Out of 479 test cases, the number of test cases solved by SynEnum is 144 test cases while SynMS is able to solve 387 test cases including 243 test cases that SynEnum cannot solve, demonstrating a substantial improvement over SynEnum. Compared to the normal rule, SynEnum solves 20 more test cases, and SynMS solves 24 more



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 6: Comparison of the running time of synthesizing winning formula and strategy for Chomp game under normal rule

test cases, with these additional test cases mainly concentrated in the T,d-I-Mark-Game category. This is because, under the misè rule, the winning formulas for these test cases become simplified. Notably, SynMS can solve all the test cases that SynEnum can, reflecting similar performance to the normal rule. For the test cases commonly solved in the Subtraction game, SynMS had an average running time of 14.87 seconds, while SynEnum took 220.31 seconds. Although SynEnum performed worse than SynMS for this group of test cases, it outperformed SynMS on 66 simpler test cases from the Take-away and T,d-I-Mark-Game categories. Further analysis of Figure 8(a) shows that SynEnum

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**30**(0)	**30**(0)	**11.14**(**10.47**)	18.71(18.71)	**6.97**(**6.97**)	8.97(8.97)
Subtraction	385	78(0)	**305**(227)	220.31(220.31)	**98.08**(**14.87**)	**10.90**(**10.90**)	68.82(24.28)
T,d-I-Mark-Game	64	36(0)	**52**(16)	**1.61**(**1.61**)	3.22(3.43)	**6.00**(**6.00**)	8.00(8.00)
Total	479	144(0)	**387**(243)	122.06(122.70)	**79.34**(**12.77**)	**8.85**(**8.87**)	56.13(17.08)

Table 14: Results of synthesizing winning formulas for Sub games under misère rule

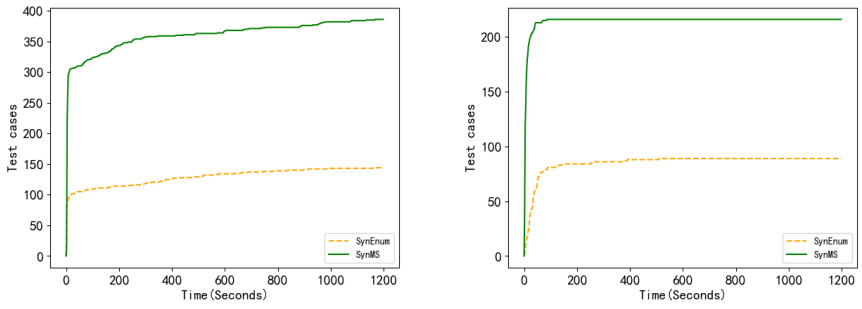
and SynMS successfully solved 99 test cases within 20 seconds, including almost all commonly solved test cases from the Take-away and T,d-I-Mark-Game cate-

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Take-away	30	**11**(5)	6(0)	31.37(10.28)	**2.45**(**2.45**)	63.55(**41.33**)	**51.17**(51.17)
Subtraction	385	42(8)	**180**(146)	52.14(58.45)	**8.19**(**2.16**)	**33.14**(**32.56**)	231.16(107.24)
T,d-I-Mark-Game	64	**36**(21)	30(15)	57.24(82.04)	**8.41**(**10.06**)	66.17(**50.07**)	**42.87**(50.20)
Total	479	89(34)	**216**(161)	51.64(59.63)	**8.06**(**4.35**)	**50.26**(**38.29**)	200.0(85.56)

Table 15: Results of synthesizing winning strategies for Sub games under misère rule

gories. After 20 seconds, the test cases that SynEnum continued to solve mainly involved more complex winning formulas synthesis problems.

Table 15 and Figure 8(b) summarize the results of synthesizing winning strategies. SynEnum successfully solved 89 test cases, including 34 test cases with larger arithmetic atoms that SynMS could not solve. For the test cases commonly solved, SynMS typically had a shorter average running time than SynEnum. Regarding strategy size, SynMS typically produced larger strategies compared to SynEnum, which aligns with the results of synthesizing the winning formulas.



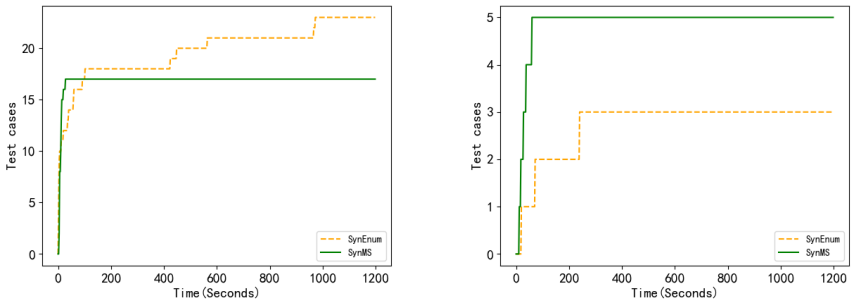
(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 7: Comparison of the running time of synthesizing winning formula and strategy for Sub game under misère rule

Comparison on Nim game. Table 16 and Table 17 presents the results about synthesizing winning formulas and strategies under the misère rule in the

Nim game. Compared to the normal rule, the winning formulas of the test cases in the game become more complex. As a result, both SynEnum and SynMS saw a decrease in the number of solved test cases about synthesizing winning formulas. Compared to the normal rule, SynEnum solved 23 test cases, 43 fewer than before, with a decrease in the solve rate of 14.23%. SynMS faced a more severe situation, solving only 17 test cases, 56 fewer than before, with a decrease in the solve rate of 18.54%. An in-depth analysis of Figure 8(a) shows that, although SynMS solved fewer test cases than SynEnum, it is able to synthesize 17 test cases within the first 28 seconds, while SynEnum could only synthesize 12 test cases. After 28 seconds, SynEnum shows greater advantages in solving test cases with winning formulas consisting of larger arithmetic atoms. Moreover, in the synthesis of winning strategies, SynMS showed a clear advantage in synthesizing complex strategies. SynMS is able to synthesize all 3 test cases that SynEnum successfully solved and also solved 2 test cases that SynEnum can not synthesize. Figure 8(b) clearly shows that SynMS outperforms SynEnum in synthesis efficiency at all times. For the test cases solved by both approaches, the comparison of the average formula and strategy size are similar to those observed in the previous games.



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 8: Comparison of the running time of synthesizing winning formula and strategy for Nim game under misère rule

Comparison on Welter game. The results about synthesizing winning

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	**1**(0)	**1**(0)	422.33(422.33)	**5.35**(**5.35**)	**20.0**(**20.0**)	130.0(130.0)
Monotonic Nim	7	**2**(1)	1(0)	**1.83**(**1.50**)	13.38(13.38)	10.00(9.00)	**8.00**(**8.00**)
Empty and Transfer	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	**2.30**(**2.30**)	7.8(10.42)	**10.00**(**10.00**)	128.50(42.00)
l-Slow Nim	180	**7**(5)	2(0)	37.28(**3.83**)	**6.84**(6.84)	**9.57**(**8.50**)	88.50(88.50)
Sharing Nim	7	**1**(0)	**1**(0)	39.87(39.87)	**26.77**(**26.77**)	**17.00**(**17.00**)	23.00(23.00)
Small Nim	7	**2**(1)	1(0)	495.75(19.77)	**3.25**(**3.25**)	19.00(**18.00**)	**18.00**(**18.00**)
Greedy Nim	7	**1**(0)	**1**(0)	562.30(562.30)	**5.09**(**5.09**)	**20.00**(**20.00**)	130.00(130.00)
Odd-or-Even Nim	7	0(0)	**1**(1)	-(-)	18.30(-)	-(-)	295.00(-)
Modular Nim	5	**1**(0)	**1**(0)	**1.85**(**1.85**)	4.06(4.06)	**8.00**(**8.00**)	25.00(25.00)
Pointed Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
End Nim	7	**1**(0)	**1**(0)	100.50(100.50)	**2.82**(**2.82**)	**18.00**(**18.00**)	**18.00**(**18.00**)
Loop End Nim	7	**1**(0)	**1**(0)	965.87(965.87)	**10.92**(**10.92**)	**20.00**(**20.00**)	79.00(79.00)
Staircase Nim	7	**2**(1)	1(0)	**1.26**(**0.41**)	13.68(13.68)	**5.00**(**4.00**)	6.00(6.00)
Circular Nim	15	**3**(0)	**3**(0)	149.52(149.52)	**8.10**(**8.10**)	**10.00**(**10.00**)	65.00(65.00)
Total	302	**23**(8)	17(2)	165.31(171.53)	**9.25**(**8.92**)	**12.09**(**12.73**)	80.05(56.73)

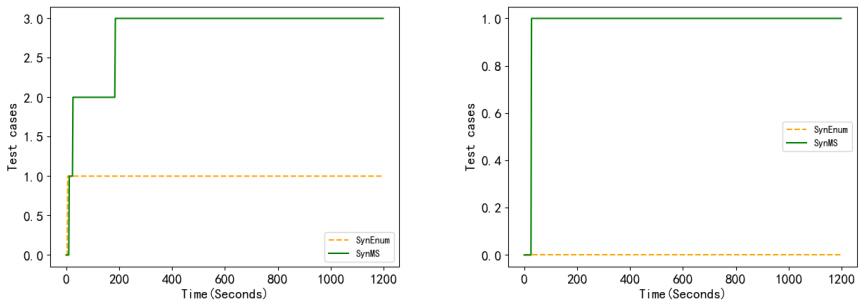
Table 16: Results of synthesizing winning formulas for Nim games under misère rule

formulas and strategies are shown in Table 18, 19, and Figures 9. Similar to the case with Nim, the Welter game becomes more complex under the misère rule, resulting in a decrease in the number of solved test cases for both SynEnum and SynMS. In terms of synthesizing winning formulas, SynEnum solved only 1 test case, while SynMS is able to solve only 3 test cases. For the 1 test case solved by both, the running times of SynEnum and SynMS are almost identical, both within a very short time of 12 seconds. However, in the synthesis of winning strategies, SynEnum is unable to solve any test cases, while SynMS successfully solved only 1 test case. This change is also clearly evident in Figure 9.

Comparison on Wythoff game. Table 20 presents the results about synthesizing winning formulas under the misère rule in the Wythoff game. Similar

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Monotonic Nim	7	0(0)	**1**(1)	-(-)	59.72(-)	-(-)	96.0(-)
Empty and Transfer	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty All But One	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Empty and Redistribute	6	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Entropy Reduction Game	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Min Game	6	1(0)	**2**(1)	18.70(18.70)	**13.77**(**17.32**)	**39.00**(**39.00**)	232.5(70.00)
l-Slow Nim	180	**1**(0)	**1**(0)	239.23(239.23)	**37.23**(**37.23**)	**120.00**(**120.00**)	173.00(173.00)
Sharing Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Small Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Greedy Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Odd-or-Even Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Modular Nim	5	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Pointed Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
End Nim	7	**1**(0)	**1**(0)	70.36(70.36)	**26.25**(**26.25**)	149.00(149.00)	**74.00**(**74.00**)
Loop End Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Staircase Nim	7	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Circular Nim	15	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	302	3(0)	**5**(2)	109.43(109.43)	**30.15**(**26.93**)	**102.67**(**102.67**)	161.6(105.67)

Table 17: Results of synthesizing winning strategies for Nim games under misère rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 9: Comparison of the running time of synthesizing winning formula and strategy for Welter game under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	**1**(1)	-(-)	185.35(-)	-(-)	1166.00(-)
I_welter Game	5	**1**(0)	**1**(0)	**5.80**(**5.80**)	11.70(11.70)	**8.00**(**8.00**)	9.00(9.00)
Max_welter Game	1	0(0)	**1**(1)	-(-)	25.26(-)	-(-)	97.00(-)
Total	7	1(0)	**3**(2)	**5.80**(**5.80**)	74.10(11.70)	**8.00**(**8.00**)	424.00(9.00)

Table 18: Results of synthesizing winning formulas for Welter games under misère rule

Category	Test cases	Strategy					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
I_welter Game	5	0(0)	**1**(1)	-(-)	27.87(-)	-(-)	115.00(-)
Max_welter Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
Total	7	0(0)	**1**(1)	-(-)	27.87(-)	-(-)	115.00(-)

Table 19: Results of synthesizing winning strategies for Welter games under misère rule

to the cases with Nim and Welter, a similar situation also occurs in the synthesis of winning formulas in the Wythoff game. SynEnum can solve $1,416$ test cases, solving 321 fewer than before. SynMS can solve $2,185$ test cases, solving 102 fewer than before. These test cases are mainly concentrated in games like Monotonic_l_Diet_Game, st1_lm_restricted, and nd2_lm_restricted. Although both SynEnum and SynMS saw a decrease in the number of solved test cases, the number of solved test cases by both remained above $1,400$. Under the normal rule, both SynEnum and SynMS are able to solve over $1,200$ test cases within 5 seconds. Similarly, under the misère rule, they are also able to solve more than $1,000$ test cases within 5 seconds as shown in Figure 10(a). Similar to its performance in Nim and Welter games, SynEnum solved 203 fewer test cases under the misère rule compared to the normal rule, primarily in the rd2_lm_restricted, rd3_lm_restricted, and th4_lm_restricted games. In these games, these test cases are more difficult for SynEnum to solve under the misère rule. Additionally, its performance in terms of runtime, average formula

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	7(0)	**17**(10)	**5.06**(**5.06**)	29.97(6.41)	**10.00**(**10.0**)	155.71(36.86)
l_Wythoff_Game	140	55(0)	**89**(34)	62.45(62.45)	**51.39**(**22.78**)	**11.42**(**11.42**)	117.51(61.35)
Monotonic_l_Diet_Game	105	44(0)	**67**(23)	43.25(43.25)	**25.04**(**4.57**)	**8.89**(**8.89**)	75.25(37.91)
Wythoff with subtraction set S	315	158(0)	**260**(102)	57.89(57.89)	**26.56**(**7.06**)	**10.68**(**10.68**)	142.43(63.65)
Wythoff with a function f	168	82(6)	**120**(44)	99.53(73.69)	**18.72**(**11.14**)	**11.93**(**11.54**)	118.65(60.33)
1st(l_1,l_2)-restricted Wythoff	315	138(1)	**243**(106)	122.17(122.4)	**53.46**(**12.78**)	**10.93**(**10.91**)	154.37(65.24)
2nd(l_1,l_2)-restricted Wythoff	600	365(22)	**494**(151)	73.65(76.78)	**32.04**(**17.28**)	**10.77**(**10.50**)	99.25(61.41)
3rd(l_1,l_2)-restricted Wythoff	700	271(3)	**426**(158)	67.77(61.44)	**41.12**(**12.68**)	**11.61**(**11.51**)	108.24(57.09)
4th(l_1,l_2)-restricted Wythoff	560	296(1)	**469**(174)	75.04(74.62)	**34.37**(**6.17**)	**11.37**(**11.37**)	125.18(67.67)
Total	2924	1416(33)	**2185**(802)	75.57(73.54)	**35.87**(**11.83**)	**11.09**(**10.98**)	119.35(61.61)

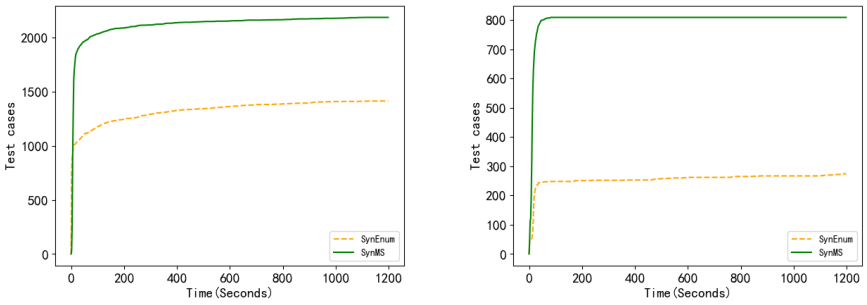
Table 20: Results of synthesizing winning formulas for Wythoff games under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Wythoff	21	1(0)	**6**(5)	17.80(17.80)	**17.27**(**12.15**)	**66.00**(**66.00**)	378.83(50.00)
l_Wythoff_Game	140	10(0)	**44**(34)	18.07(18.07)	**15.91**(**7.71**)	**62.80**(**62.80**)	454.02(81.00)
Monotonic_l_Diet_Game	105	29(0)	**61**(32)	79.64(79.64)	**10.74**(**9.08**)	**48.66**(**48.66**)	172.59(92.86)
Wythoff with subtraction set S	315	19(1)	**81**(63)	178.89(159.71)	**15.30**(**10.72**)	**80.11**(**76.22**)	402.47(178.17)
Wythoff with a function f	168	12(0)	**54**(42)	16.85(16.85)	**12.51**(**8.78**)	**63.33**(**63.33**)	359.56(75.83)
1st(l_1,l_2)-restricted Wythoff	315	18(1)	**55**(38)	167.62(176.09)	**13.88**(**10.53**)	**75.94**(**76.53**)	355.47(141.59)
2nd(l_1,l_2)-restricted Wythoff	600	94(0)	**112**(18)	103.05(103.05)	**2.09**(**1.98**)	**40.96**(**40.96**)	158.26(138.96)
3rd(l_1,l_2)-restricted Wythoff	700	50(1)	**216**(167)	**17.23**(17.21)	20.70(**8.75**)	**61.24**(**61.29**)	497.74(79.69)
4th(l_1,l_2)-restricted Wythoff	560	41(0)	**180**(139)	44.25(44.25)	**9.83**(**10.50**)	**62.12**(**62.12**)	488.89(71.80)
Total	2924	274(3)	**809**(538)	78.42(77.20)	**13.12**(**6.92**)	**55.52**(**55.12**)	392.56(110.66)

Table 21: Results of synthesizing winning strategies for Wythoff games under misère rule

size, and strategy size is similar to that under the normal rule.

Comparison on Chomp game. Table 22 and 23 presents the results of synthesizing winning formulas under the misère rule in the Wythoff game. Both SynEnum and SynMS are only able to solve 2 test cases. For these 2 test cases commonly solved, the average running time for SynEnum is 99.86, while the average running time for SynMS is 16.48, showing a significant reduction in



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 10: Comparison of the running time of synthesizing winning formula and strategy for Wythoff game under misère rule

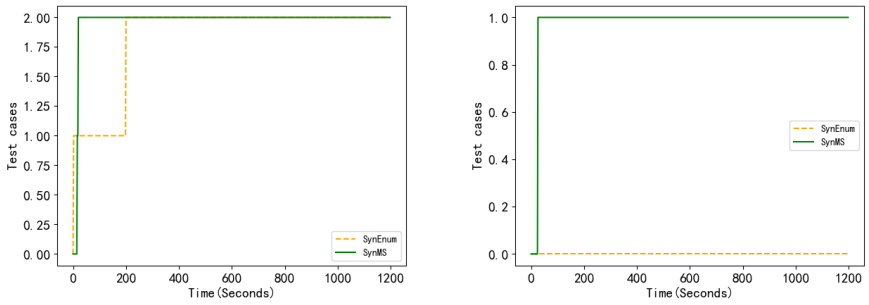
time. As for the size of formulas, the average size of SynMS's formulas is still larger than that of SynEnum. In terms of synthesizing winning strategies, as the game's winning strategies gradually become more complex, SynEnum is unable to solve any test cases, while SynMS was only able to solve 1 test case. Figure 11 illustrates this change.

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	**1**(0)	**1**(0)	**0.99**(**0.99**)	14.42(14.42)	**6.00**(**6.00**)	8.00(8.00)
L_shape Chomp Game	1	**1**(0)	**1**(0)	198.73(198.73)	**18.54**(**18.54**)	**16.00**(**16.00**)	101.00(101.00)
total	8	**2**(0)	**2**(0)	99.86(99.86)	**16.48**(**16.48**)	**11.00**(**11.00**)	54.50(54.50)

Table 22: Results of synthesizing winning formulas for Chomp games under misère rule

Category	Test cases	Formula					
		Solved		Time		Size	
		SynEnum	SynMS	SynEnum	SynMS	SynEnum	SynMS
Chomp game	7	0(0)	**1**(1)	-(-)	25.68(-)	-(-)	126.00(-)
L_shape Chomp Game	1	0(0)	0(0)	-(-)	-(-)	-(-)	-(-)
total	8	0(0)	**1**(1)	-(-)	25.68(-)	-(-)	126.00(-)

Table 23: Results of synthesizing winning strategies for Chomp games under misère rule



(a) Comparison on synthesizing winning formulas on the whole benchmark

(b) Comparison on synthesizing winning strategies on the whole benchmark

Figure 11: Comparison of the running time of synthesizing winning formula and strategy for Chomp game under misère rule

8. Related work

8.1. Analysis of ICGs

Fraenkel [29] introduced an LIA-definable ICG, namely Mark game, and showed that its set of winning states is not ultimately periodic. It follows Theorem 2 that the set of winning states is not LIA-definable. Mark game is a variant of Take-away game and shares the same constraint $\mathcal{C}$ and ending condition $\mathcal{E}$. In this game, the player can take either 1 or $\lfloor \frac{v}{2} \rfloor$ chips from the table, where v denotes the number of remaining chips. The precondition and effect of action $take(k)$ in Mark game are as follows.

- $\texttt{pre}(take(k)) : v \geq 0 \wedge k \geq 1 \wedge (k = 1 \vee v = 2k \vee v = 2k + 1)$;
- $\texttt{eff}(take(k)) : \{\top, v, v - k\}$.

Subtraction game [34] is a generalization of the Take-away game. Berlekamp et al. [38] proved that any Subtraction game admits an LIA-definable set of winning states. Let S be a finite set of positive integers. In this game, each player can take k chips from the pile where $k \in S$. The precondition and effect of action $take(k)$ in Subtraction game are as follows.

- $\texttt{pre}(take(k))$: $(\bigvee_{s \in S} k = s) \wedge v \geq \min(S)$ where $\min(S)$ denotes the smallest integer of S;
- $\texttt{eff}(take(k)) : \{\langle \top, v, v - k \rangle\}$;

Larsson and Wästlund [54] proved that the problem of deciding whether two LIA-definable ICGs have the same set of winning states is undecidable. Larsson [55] showed that the problem of deciding whether the set of winning states for an LIA-definable ICG is 2-dimensional periodic is undecidable.

8.2. Automatic methods to two player games

Two classical methods for solving the winning strategy of finite-state ICGs are backward induction [56] and alpha-beta pruning [57, 58]. Backward induction reasons backward from the ending state to the given initial state. For games with large state spaces, the computational complexity of backward induction increases rapidly, leading to low algorithmic efficiency and making it difficult to handle complex games. Alpha-beta pruning is a well-known pruning method for minimax trees. It reduces the number of nodes in the search tree that captures the plays from the given initial state by two players. However, the effectiveness of the alpha-beta pruning method depends on the traversal order of the nodes. If the traversal order is suboptimal, the effectiveness of pruning will be impacted.

However, there exists a different theoretical tool that is orthogonal to minimax search: the Sprague-Grundy theorem. This theory was built upon Bouton's complete mathematical analysis of the game of Nim [5], and was independently generalized to all short impartial games by Sprague [59] and Grundy [60]. The theory assigns a non-negative integer value to every game position s, known as the Sprague-Grundy value (or nimber). This value $G(s)$ is defined recursively as the "Minimum Excluded value" (mex) of the set of SG-values of all its successor positions $t \in N(s)$: $G(s) = \text{mex}(\{G(t) : t \in N(s)\})$. The utility of the SG-value is twofold. First, it directly determines the outcome: a position is a losing position if and only if its SG-value is 0. Second, it allows for the efficient analysis of decomposable games. A game s that is a sum of independent sub-games

$s_1, \ldots, s_k$ has a total SG-value equal to the "nim-sum" (bitwise XOR, denoted by $\oplus$) of its components: $G(s) = G(s_1) \oplus \cdots \oplus G(s_k)$. Despite the completeness of this theory, computing SG-values for a single, non-decomposable game by direct application of the mex definition still requires a deep recursive tree search, which can be computationally intensive. Therefore, how to efficiently prune this SG-value search tree has become a significant research problem. To address this computational challenge, several game-agnostic optimization algorithms have been proposed. [61] computes the SG-value $G(s)$ by sequentially testing $n = 0, 1, 2, \ldots$. It verifies if $G(s) = n$ by checking if the corresponding *sum of games* $s + \langle n \rangle_{nim}$ is a losing position, based on the properties of the nim-sum. [57] provides a different optimization strategy. They directly adapt the principles of $\alpha - \beta$ pruning to the mex function. Their algorithms (e.g., `cut`, `scut`) prune the search tree by passing a set of potential SG-values down the tree and cutting off branches. This method also successfully incorporates standard search optimizations like Enhanced Transposition Cutoff (ETC).

Recent research has also begun to bridge SG-value computation with machine learning. [62] shows they struggle to learn mathematically "simpler" impartial games like Nim. This difficulty is attributed to the challenge neural networks face in capturing the underlying mathematical structure required for SG-value computation. In response, the paper proposed an AlphaZero-inspired model incorporating game history that successfully mastered Nim, connecting classical combinatorial game theory with modern RL-based policy search.

Furthermore, an important line of research combines SG-value with $\alpha - \beta$ search, particularly in decomposable positions. [63] pioneered "Decomposition Search," applying it to Go endgames. This method uses the SG theorem to analyze independent sub-games separately, using the computed nimbers for heuristic evaluation, move ordering, or pruning within an $\alpha - \beta$ search. Recently, [64] employed a similar strategy to solve the game of Cram, combining combinatorial game theory (CGT) with an $\alpha - \beta$ solver, transposition tables, and SG-value endgame databases.

In contrast to these game-agnostic computational methods, another large body of research focuses on analyzing the SG-values of specific games to find analytical solutions or mathematical properties. Examples include analyses of the SG-function for the game of Euclid [65, 66], Wythoff's game [67], and Grundy's game [68], as well as studies on the periodicity of nim-values in games like peg duotaire [69]. However, these game-specific analytical approaches rely on deep mathematical insights into each game's rules, whereas this paper aims to provide a more general framework for automatic synthesis based on LIA formalization, which does not depend on such game-specific pre-analysis.

Luo and Liu [70] extended the situation calculus for formalizing ICGs. Situation Calculus is a formal logical language used to describe and reason about dynamic systems, particularly suitable for expressing reasoning problems related to actions and changes. They proposed a verification approach for the correctness of winning strategies that are defined in finite state automaton (FSA). The key idea of this approach is to find an invariant for the FSA strategy, which is similar to the concept of a winning formula. It is important to note that their approach only verifies whether a given FSA is a winning strategy, but it does not address the strategy synthesis problem.

Later, they [71] developed a CEGIS approach for synthesizing invariant strategies. An invariant strategy is of the form $\phi?;\pi a.a;\varphi?$, meaning whenever ϕ holds, the player can execute an action to enforce φ, and whenever φ holds, executing any action makes φ true regardless of what the action chooses. In fact, the formulas ϕ and φ are the winning and the losing formulas, respectively. Strictly speaking, Luo and Liu [71]'s approach only constructs winning and losing formulas but not winning strategies that specifies which action to do in different partitions of winning states (cf. Definition 4). In addition, as reported in [71], their approach is much less efficient than Enum. Hence, we do not compare with their approach in experimental evaluation.

8.3. LTL synthesis

Pnueli and Rosner [72] first proposed a well-known class of games, namely

linear temporal logic (LTL) games, where the game is represented by an automaton, and the goal is expressed by a propositional LTL formula. The objective of LTL synthesis is to synthesize a strategy that ensures goal achievement. These games are represented by an automaton, and the goals are expressed by an LTL formula. An LTL formula consists of a set of atomic propositions, logical operators, and temporal operators. The method proposed in [72] is an automaton-based approach causing a doubly exponential time complexity. To lower the complexity, Asarin et al. [73] restrict the goal to be some fragments of LTL formulas so as to obtain polynomial synthesis approaches, *e.g.*, the safety and reachability games whose goal is of the form $\Box\phi$ and $\Diamond\phi$ respectively where ϕ is propositional. The LTL synthesis problem has been addressed in numerous literature, such as [74, 75, 76, 77, 78, 79]. The above methods only focus on games over finite states.

In general, it is undecidable for the synthesis problem of infinite-state games [80]. Some decidable restricted classes of infinite-state games are identified, *e.g.*, games on automatic graphs [81], pushdown graphs [82, 83], and prefix-recognizable graphs [84].

Besides automatons, arithmetic formulas are an alternative to formalization for games. Compared to the former, the latter are easier to understand, and hence being more suitable for human beings to give a complete description. Recently, arithmetic formulas are used to represent infinite-state games [85, 86, 87, 88]. Our representative framework is also based on arithmetic formulas, and close to their work.

Beyene et al. [85] devise a sound and relatively complete proof rules for computing winning strategies for safety, reachability and LTL games over infinite states. All impartial combinatorial games considered in this paper are a reachability game over infinite states. The rules for reachability games rely on winning formulas and well-founded relations that are used to guarantee the correctness and termination of the winning strategy respectively. The generation of winning formulas and well-founded relations is solved by the EHSF solver [89]. Our approach finds the winning formula via an enumerative approach,

and does not generate the well-founded relation as an impartial combinatorial game always ends in a finite number of moves. One major drawback of Beyene et al.'s approach is that it requires users to provide hints in the form of templates to synthesize winning strategies, and hence it is not a full mechanization. By contrast, our approach succeeds in computing winning strategies for all games considered in this paper in a fully automated way.

Farzan and Kincaid [86] propose a fully automated method, called SimSynth, for synthesizing winning strategies for reachability games over infinite states. SimSynth firstly unrolls games as a bounded variation. Then it reduces the problem of synthesizing the winning strategy for the bounded variation to the linear arithmetic satisfiability problem [90]. Finally, it attempts to generalize the strategy so as to hold for all states. The insight of our approach is different from SimSynth. Instead, we first construct the winning formula, and then generate the winning strategy from the cover of the winning formula. SimSynth does not involve the winning formula, which precisely describes the set of winning states. In practice, SimSynth is only able to synthesizes the winning strategy of an ICG instance, and does not scale up well. For example, it fails to solve the 2-piled Nim game for the state $v_1 = 6 \wedge v_2 = 6$ within 10 minutes. By contrast, our approach succeeds in computing generalized winning strategies, which works for not only one state but also all winning states, for the 2-piled Nim game in 3.91 seconds.

Baier et al. [87] presents a causality-based algorithm for solving two-player reachability games defined by logical constraints. The approach leverages Craig interpolation to identify necessary subgoals—critical game slices the reachability player must traverse—and recursively decomposes the game along these subgoals. This enables the extraction of structured winning strategies. Recently, Faella and Parlato [88] consider introduces a special class of logical reachability games, called GMTs (logical reachability games modulo theories), where the safety player has a limited number of moves per configuration, while the reachability player has no such restriction. The authors show that GMTs can model real-world problems, such as the Cinderella-Stepmother game, and demonstrate how determining the winner in these games can be reduced to solving a system

of constrained Horn clauses (CHCs). As shown in [88], the running times are significantly lower compared to implementations of Baier et al. [87].

But these approaches are not suitable for ICGs. In addition to the reasons mentioned above, there are more reason as follows: Firstly, these approaches only generates a winning strategy for a specific initial state, and the generated strategy cannot be generalized to all legal states. Additionally, the formalization of these approaches are based on linear real arithmetic (LRA) rather than LIA. The differences between LIA and LRA are the following: (1) variables in LIA have an integer domain rather than real one; and (2) LIA contains the congruence modulo relation while LRA does not. The majority of ICGs contain only integer variables and require congruence modulo relation in their winning formula.

Previous research has used LIA-based formalization, but it only supports safety games [91]. Daniel and Oliver [91] proposes a machine learning framework for synthesizing reactive controllers in safety games, particularly those defined over infinite or prohibitively large graphs. The core technique involves a feedback loop between a symbolic teacher component and a learning algorithm, where the learning algorithm learns a winning set, an approximation of the vertices from which the system player can win. The learning process iteratively refines this set using counterexamples provided by the teacher. The paper also introduces a decision tree learning algorithm tailored for this setting, ensuring the efficient learning of winning sets. However, the approaches can only compute an approximate winning set starting from a given state, meaning the winning set does not fully cover all possible winning states, and the corresponding winning strategy does not cover all winning paths.

Recent work on strategy synthesis for infinite state systems has developed several complementary directions. One direction studies automata learning. Neider and Topcu [81] proposed a query based framework for synthesizing safety controllers over infinite graphs. Their method learns a finite automaton for the winning region by interacting with a symbolic teacher and refining the hypothesis with counterexamples. This approach is effective for safety objectives, but it relies on teacher support and regular representations of the state space. There-

fore, it is not directly suited to our setting, where we aim to synthesize arithmetic winning strategies for all winning states in LIA definable ICGs.

Another direction studies symbolic fixed point methods for temporal objectives. Samuel et al. [92] developed symbolic fixed point algorithms for logical LTL games and showed that classical fixed point reasoning can be extended to symbolic infinite state settings. Their method can handle a broad class of temporal objectives without requiring user supplied templates. However, its output is centered on symbolic controllers rather than explicit arithmetic rules over partitions of the winning region. Moreover, when the game contains rich arithmetic structure, such as modular constraints that are common in ICGs, the fixed point computation may still be expensive.

A related direction improves symbolic game solving by introducing loop summaries that help fixed point iterations converge. Heim and Dimitrova [93] showed how such summaries can be used to solve infinite state games with objectives including reachability, safety, Büchi, and co Büchi conditions. This is an important optimization for symbolic solving, especially when naive fixed point iteration diverges. For our purpose, however, the method is less direct, because it does not focus on constructing an explicit generalized rule based strategy that covers all winning states.

Schmuck et al. [94] further improve scalability through localized attractor computation. Their method uses finite abstractions to identify useful subgames, computes local information there, and reuses the results while solving the full game. This idea is effective and technically appealing, but it depends on the quality of the abstraction and on the availability of useful local decompositions. As a result, it is less suitable for ICGs, whose winning strategies are more naturally described by global arithmetic conditions over integer variables.

Azzopardi et al. [95] study full LTL synthesis over infinite state arenas through abstraction refinement. Their framework is attractive in its generality and can handle full temporal specifications. However, this generality also brings a cost: the approach depends heavily on predicate discovery and refinement, and it may face a rapid growth of the abstract state space. In addition, these meth-

ods are designed for more general temporal synthesis problems, rather than for synthesizing interpretable winning formulas and winning strategies tailored to the structure of impartial combinatorial games.

8.4. Generalized planning

A related problem is generalized planning that aims to construct a generalized planning that works for possibly infinitely many planning instances sharing the similar structure [96]. Levesque [96] implemented an iterative planner called KPLANNER. The core idea is to first search for planning solutions with no loop structure and variable values less than a threshold, and then identify the common structure in the planning solution based on the built-in pattern matching mechanism of the Prolog language, and extract the planning program with loop structure. On this basis, Hu and Levesque [97] proposed a new planner FSAPLANNER. Compared with KPLANNER, FSAPLANNER proposed a more expressive planning solution form: finite-state automata (FSA). At the same time, they proposed that the existence of planning solutions for one-dimensional planning problems is decidable. Later, Hu and Giacomo [98] analyzed the complexity of the problem and proved that it is complete in exponential space. However, the above methods are mainly aimed at single-variable scenarios and are difficult to directly extend to complex planning problems with multiple variables.

To solve the multivariate general planning problem, Srivastava et al. [99] proposed a method to generate a FSA plan based on state abstraction using 3-valued logic. The method can be applied to loop structures, but the generated solution is not guaranteed to achieve the goal. To solve this problem, they designed an algorithm to calculate the conditions for the planning to terminate and achieve the goal when it only contains simple loop structures, namely applicability condition[100]. Nevertheless, the planners synthesized by their method are only allowed to have simple loop structures and cannot synthesize planners with nested loops.

Compiling a generalized planning problem into another planning problem and then indirectly solving the original one is also an effective method. Bonet

et al. [101] proposed a method to automatically construct a finite state controller (FSC). This method transforms a partially observable planning problem into a consistent planning problem, and then uses a classical planning solver to indirectly solve the consistent planning problem. However, it is not clear which variants of the original problem the generated FSC can solve. Bonet and Geffner [102] clarify which original problems the FSC generated by Bonet et al. [101] can solve by characterizing the general structure that supports policy generalization. Segovia et al. codify a finite number of generalized planning problems with the same problem domain into a single classical planning problem [103, 104, 105]. However, their approach is only applicable to classical planning problems. Srivastava et al. [106] proposed a new type of planning problem: Qualitative Numeric Planning (QNP). QNP imposes two restrictions on Numerical Planning: (1) Any formula is a Boolean combination of simple numeric literals in the form of $v > 0$ or $v = 0$; (2) The value of the variable decreases or increases by an uncertain amount. Under these two constraints, the state space of the QNP can be compressed to a finite space of size $2^{|V|}$, where $|V|$ is the number of numerical variables. Therefore, the QNP problem is decidable. But the expressive power of the QNP proposed above is relatively limited. Bonet et al. [107] used SAT solvers and description logic to learn an abstract QNP from a finite number of problem instances. However, the learned QNP may not be a valid abstraction for the entire set of original problem instances. Later, Bonet et al. [108] studied the necessary conditions to ensure that an abstraction is valid, that is, which problem instances can guarantee that the abstraction is valid.

The abstraction-refinement framework is an effective approach for verifying the properties of multi-agent systems. The framework first builds an abstract model based on 3-value semantics to preliminarily judge the system properties. If the property is true or false in the abstract model, the truth or falsehood of the property in the specific model can be directly determined; if it cannot be determined, it means that the abstract model is too rough and cannot accurately reflect the satisfaction of the property. At this point, the framework refines the abstract model by searching for failed states until a clear conclusion can be

drawn. In order to achieve this process, a variety of abstraction and refinement strategies have been proposed based on this framework [109, 110, 111, 112]. Nevertheless, the abstraction-refinement framework cannot guarantee a complete abstract model, because it is undecidable to find the failure state in these two logics. In addition, the above works all focus on the abstraction of the model rather than the theoretical abstraction.

In this paper, we concentrate on ICGs that is more complex than the planning domain. The former involves two players while the latter only focuses on the single-agent case. Hence it is necessary to consider not only how the current player chooses actions, but also how the other player responds when we synthesize winning strategies. Recently, De Giacomo et al. [113] study the translation from generalized planning under partial observability to two-player games with perfect information. In the case there are no additional fairness assumptions, the former corresponds to two-player games with imperfect information. By adapting the belief-state construction, De Giacomo et al. remove imperfect information from games, and hence obtain a two-player game with perfect information corresponding to the given generalized planning problem. However, they do not provide an approach to synthesizing winning strategies for the games with perfect information corresponding to the given generalized planning problem.

9. Conclusions and future work

We have devised an effective approach to computing generalized winning strategies that works for possibly infinite many instances of ICGs. The key insight of our approach is to generate a candidate formula that is consistent with two generated sets of positive states and negative states. To this end, we have put forward two approach to solve this problem. One adapts an enumerative method, and the other improves efficiency by reducing the synthesis problem as an exact binary classification. Then, by integrating the SMT-based verification procedure, we have developed a CEGIS approach to synthesizing the winning formula and strategy of ICGs. We have implemented our approach and experi-

mental results show the effectiveness and scalability of our proposed approach.

Our approach have some limitations, and hence leading to several avenues for future work. Firstly, the ICGs considered in this paper are formalized in LIA. For other ICGs, their formalization involves real numbers, existential quantifiers, multiple operator, rounding operator, exclusive-disjunction operator and so on. We would also like to consider the games whose formalizations require more expressive language. Secondly, for the same solvable test cases, the strategy generated based on exact binary classification is larger than the one generated by the enumeration method, while the winning strategy can be represented in a more concise form. We would like to design the solution to strategy synthesis based on more efficient algorithms. Finally, we focus on ICGs. It is also interesting to apply our approach to other games, *e.g.*, partizan combinatorial games (*i.e.*, some moves are available to one player and not to the other).

References

[1] K. Chatterjee, T. A. Henzinger, N. Piterman, Strategy logic, Information and Computation 208 (2010) 677–693.

[2] D. Zhang, M. Thielscher, Representing and Reasoning about Game Strategies, Journal of Philosophical Logic 44 (2015) 203–236.

[3] L. Xiong, Y. Liu, Strategy Representation and Reasoning for Incomplete Information Concurrent Games in the Situation Calculus, in: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 1322–1329.

[4] T. S. Ferguson, Game Theory, World Scientific, 2018.

[5] C. L. Bouton, Nim, a Game with a Complete Mathematical Theory, Annals of Mathematics 3 (1901) 35–39.

[6] W. A. Wythoff, A modification of the game of nim, Nieuw Archief voor Wiskunde 7 (1907) 199–202.

[7] F. Schuh, The Game of Divisions, Nieuw Tijdschrift voor Wiskunde 39 (1952) 299–304.

[8] T. S. Ferguson, Some Chip Transfer Games, Theoretical Computer Science 191 (1998) 157–171.

[9] W. Liu, M. Zhao, General Restrictions of Wythoff-like Games, Theoretical Computer Science 602 (2015) 80–88.

[10] J. Ahn, B. Chen, R. Chen, E. Erives, J. Fleming, M. Gerovitch, T. Gopalakrishna, T. Khovanova, N. Malur, N. Polina, P. Sahoo, On Variations of Nim and Chomp, Minnesota Journal of Undergraduate Mathematics 4 (2018) 1–11.

[11] S. W. Golomb, A mathematical investigation of games of “take-away", Journal of Combinatorial Theory 1 (1966) 443–458.

[12] G. Beaulieu, K. Burke, E. Duchêne, Impartial Coloring Games, Theoretical Computer Science 485 (2013) 49–60.

[13] E. Sopena, i-Mark: A new subtraction division game, Theoretical Computer Science 627 (2016) 90–101.

[14] V. Gurvich, N. B. Ho, On tame, pet, domestic, and miserable impartial games, Discrete Applied Mathematics 243 (2018) 54–72.

[15] D. de Jonge, D. Zhang, Lifted Backward Search for General Game Playing, in: Proceedings of the Twenty-Ninth Australasian Joint Conference on Advances in Artificial Intelligence (AI-2016), 2016, pp. 3–16.

[16] M. Fox, D. Long, PDDL2.1 : An Extension to PDDL for Expressing Temporal Planning Domains, Journal of Artificial Intelligence Research 20 (2003) 61–124.

[17] A. Udupa, A. Raghavan, J. V. Deshmukh, S. Mador-Haim, M. M. K. Martin, R. Alur, TRANSIT: Specifying Protocols with Concolic Snippets, in:

Proceedings of the Thirty-Fourth Annual ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI-2013), 2013, pp. 287–296.

[18] L. D. Moura, N. Bjørner, Z3: an Efficient SMT Solver, in: Proceedings of the Fourteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2008), volume 4963 of *Lecture Notes in Computer Science*, Springer, 2008, pp. 337–340.

[19] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanovic, T. King, A. Reynolds, C. Tinelli, CVC4, in: Proceedings of the Twenety-Third International Conference on Computer Aided Verification (CAV-2011), volume 6806 of *Lecture Notes in Computer Science*, Springer, 2011, pp. 171–177.

[20] A. Cimatti, A. Griggio, B. J. Schaafsma, R. Sebastiani, The MathSAT5 SMT Solver, in: Proceedings of the Nineteenth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2013), volume 7795 of *Lecture Notes in Computer Science*, Springer, 2013, pp. 93–107.

[21] K. Wu, L. Fang, L. Xiong, Z. Lai, Y. Qiao, K. Chen, F. Rong, Automatic Synthesis of Generalized Winning Strategies of Impartial Combinatorial Games Using SMT Solvers, in: Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-2020), 2020, pp. 1703–1711.

[22] L. Fang, M. Yang, D. Cheng, Y. Hao, Q. Guan, L. Xiong, Generalized Strategy Synthesis of Infinite-State Impartial Combinatorial Games via Exact Binary Classification, in: Proceedings of the Twenty-Third International Conference on Autonomous Agents and Multiagent Systems (AAMAS-2024), 2024, pp. 562–570.

[23] D. C. Cooper, Theorem Proving in Arithmetic without Multiplication, Machine Intelligence 7 (1972) 300.

[24] D. Monniaux, Quantifier Elimination by Lazy Model Enumeration, in: Proceedings of the Twenty-Second International Conference on Computer Aided Verification (CAV-2010), volume 6174 of *Lecture Notes in Computer Science*, Springer, 2010, pp. 585–599.

[25] S. Ginsburg, E. Spanier, Semigroups, Presburger formulas, and languages, Pacific Journal of Mathematics 16 (1966) 285–296.

[26] A. Cobham, On the base-dependence of sets of numbers recognizable by finite automata, Mathematical Systems Theory 3 (1969) 186–192.

[27] E. Zermelo, Über eine anwendung der mengenlehre auf die theorie des schachspiels, in: Proceedings of the Fifth International Congress of Mathematicians Conference (ICM-1913), 1913, pp. 501–504.

[28] D. Gale, F. M. Stewart, Infinite games with perfect information, in: H. W. Kuhn, A. W. Tucker (Eds.), Contributions to the Theory of Games, volume 2 of *Annals of Mathematics Studies*, Princeton University Press, 1953, pp. 245–266.

[29] A. S. Fraenkel, Aperiodic subtraction games, Electronic Journal of Combinatorics 18 (2011) 1–12.

[30] J. Hoffmann, R. I. Brafman, Conformant planning via heuristic forward search: A new approach, Artificial Intelligence 170 (2006) 507–541.

[31] J. Rintanen, Planning as satisfiability: Heuristics, Artificial Intelligence 193 (2012) 45–86.

[32] M. Presburger, Über die vollständigkeit eines gewissen systems der arithmetik ganzer zahlen, in welchem die addition als einzige operation hervortritt, in: Comptes Rendus du I congrès de Mathématiciens des Pays Slaves, 1929, pp. 92–101.

[33] Y. Chu, S. Cai, C. Luo, NuWLS: Improving Local Search for (Weighted) Partial MaxSAT by New Weighting Techniques, in: Proceedings of the

Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-2023), 2023, pp. 3915–3923.

[34] I. M. Yaglom, Two games with matchsticks, Kvant Selecta: Combinatorics, I 1 (2001) 1–8.

[35] C. P. Welter, The advancing operation in a special abelian group, Indagationes Mathematicae 55 (1952) 304–314.

[36] G. Cairns, N. B. Ho, Min, A Combinatorial Game Having a Connection With Prime Numbers, Integers 10 (2010) 765–770.

[37] N. B. Ho, Three-pile Sharing Nim and the Quadratic Time Winning Strategy, Theoretical Computer Science 646 (2016) 49–60.

[38] E. R. Berlekamp, J. H. Conway, R. K. Guy, Winning Ways for Your Mathematical Plays, 2nd ed., AK Peters/CRC Press, 2001.

[39] M. Albert, R. J. Nowakowski, Nim Restrictions, Integers: Electronic Journal of Combinatorial Number Theory 4 (2004) 1–10.

[40] F. Smith, P. Stănică, Comply/Constrain Games or Games with a Muller Twist, Integers: Electronic Journal of Combinatorial Number Theory 2 (2002) 1–11.

[41] H. Gavel, P. Strimling, Nim with a Modular Muller Twist, Integers: Electronic Journal of Combinatorial Number Theory 4 (2004) 1–9.

[42] M. H. Albert, R. J. Nowakowski, The Game of End-Nim, Electronic Journal of Combinatorics 8 (2011) 1–12.

[43] R. Sprague, Über zwei abarten von nim, Tohoku Mathematical Journal 43 (1937) 351–354.

[44] M. Dufour, S. Heubach, Circular Nim Games, The Electronic Journal of Combinatorics 20 (2013) 1–26.

[45] J. Kahane, A. S. Fraenkel, k-Welter–A Generalization of Welter's Game, Journal of Combinatorial Theory, Series A 46 (1987) 1–20.

[46] N. B. Ho, The Max–Welter Game, Discrete Mathematics 318 (2014) 41–47.

[47] A. S. Fraenkel, How to beat your wythoff games' opponent on three fronts, The American Mathematical Monthly 89 (1982) 353–361.

[48] E. Duchene, A. Fraenkel, S. Gravier, R. J. Nowakowski, Another bridge between Nim and Wythoff, The Australasian Journal of Combinatorics 44 (2009) 43–56.

[49] A. S. Fraenkel, N. B. Ho, When are Translations of P-positions of Wythoff's Game P-positions?, Integers 15A (2015) 1–12.

[50] V. Gurvich, Further Generalizations of the Wythoff Game and the Minimum Excludant, Discrete Applied Mathematics 160 (2012) 941–947.

[51] W. Liu, H. Li, General Restriction of (s, t)-Wythoff's Game, The Electronic Journal of Combinatorics 21 (2014) 2–44.

[52] D. Eric, G. Sylvain, Geometrical Extensions of Wythoff's game, Discrete Mathematics 309 (2009) 3595–3608.

[53] R. Alur, A. Radhakrishna, A. Udupa, Scaling Enumerative Program Synthesis via Divide and Conquer, in: Proceedings of Twenty-Third International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2017), volume 10205 of *Lecture Notes in Computer Science*, Springer, 2017, pp. 319–336.

[54] U. Larsson, J. Wästlund, From heaps of matches to the limits of computability, The Electronic Journal of Combinatorics 20 (2013) 1–12.

[55] U. Larsson, Impartial games emulating one-dimensional cellular automata and undecidability, Journal of Combinatorial Theory, Series A 120 (2013) 1116–1130.

[56] J. Watson, Strategy: an introduction to game theory, Third ed., W. W. Norton & Company, 2002.

[57] P. Beling, M. Rogalski, On Pruning Search Trees of Impartial Games, Artificial Intelligence 283 (2020) 103262.

[58] G. T. Heineman, G. Pollice, S. Selkow, Algorithms in a Nutshell, O'Reilly Media, 2008.

[59] R. Sprague, Über mathematische kampfspiele, Tohoku Mathematical Journal, First Series 41 (1935) 438–444.

[60] P. M. Grundy, Mathematics and games, Eureka 2 (1939) 6–8.

[61] J. Lemoine, S. Viennot, Nimbers are inevitable, Theoretical Computer Science 462 (2012) 70–79.

[62] S. Riis, Mastering nim and impartial games with weak neural networks: An alphazero-inspired multi-frame approach, arXiv preprint arXiv:2411.06403 (2024).

[63] M. Müller, Decomposition search: A combinatorial games approach to game tree search, with applications to solving go endgames, in: IJCAI, 1999, pp. 578–583.

[64] J. W. Uiterwijk, Solving cram using combinatorial game theory, in: Advances in Computer Games, Springer, 2019, pp. 91–105.

[65] G. Cairns, N. B. Ho, T. Lengyel, The sprague-grundy function of the real game euclid, Discrete Mathematics 311 (2011) 457–462.

[66] G. Nivasch, The sprague-grundy function of the game euclid, Discrete Mathematics 306 (2006) 2798–2800.

[67] U. Blass, A. S. Fraenkel, The sprague-grundy function for wythoff's game, Theoretical computer science 75 (1990) 311–333.

[68] R. K. Guy, C. A. Smith, The g-values of various games, in: Mathematical Proceedings of the Cambridge Philosophical Society, volume 52, Cambridge University Press, 1956, pp. 514–526.

[69] J. Grossman, Periodicity in one-dimensional peg duotaire, Theoretical Computer Science 313 (2004) 417–425.

[70] K. Luo, Y. Liu, Automatic Verification of FSA Strategies via Counterexample-Guided Local Search for Invariants, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-2019), 2019, pp. 1814–1821.

[71] K. Luo, Y. Liu, Automated Synthesis of Generalized Invariant Strategies via Counterexample-Guided Strategy Refinement, in: Proceedings of the Thirty-Sixth AAAI Conference on Artificial Intelligence (AAAI-2022), 2022, pp. 5800–5808.

[72] A. Pnueli, R. Rosner, On the Synthesis of a Reactive Module, in: Proceedings of the Sixteenth ACM SIGPLAN-SIGACT Symposium Conference on Principles of programming languages (POPL-1989), 1989, pp. 179–190.

[73] E. Asarin, O. Maler, A. Pnueli, J. Sifakis, Controller Synthesis for Timed Automata, in: Proceedings of the Fifth International Federation of Automatic Control Conference on System Structure and Control (SSC-1998), 1998, pp. 447–452.

[74] E. Niklas, L. Alexander, N. Nina, R. Leonid, SAT-based Strategy Extraction in Reachability Games, in: Proceedings of the Twenty-Ninth International Conference on Artificial Intelligence (AAAI-2015), 2015, pp. 3738–3745.

[75] S. Zhu, L. M. Tabajara, J. Li, G. Pu, M. Y. Vardi, Symbolic LTL_f Synthesis, in: Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence (IJCAI-2017), 2017, pp. 1362–1369.

[76] A. Camacho, C. Muise, J. A. Baier, S. A. McIlraith, LTL Realizability via Safety and Reachability Games, in: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI-2018), 2018, pp. 4683–4691.

[77] G. D. Giacomo, M. Favorito, J. Li, M. Y. Vardi, S. Xiao, S. Zhu, LTL_f Synthesis as AND-OR Graph Search: Knowledge Compilation at Work, in: Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI-2022), 2022, pp. 2591–2598.

[78] M. Balachander, E. Filiot, J. Raskin, LTL Reactive Synthesis with a Few Hints, in: Proceedings of the Twenty-Ninth International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2023), volume 13994 of *Lecture Notes in Computer Science*, Springer, 2023, pp. 309–328.

[79] J. Kretínský, T. Meggendorfer, M. Prokop, S. Rieder, Guessing Winning Policies in LTL Synthesis by Semantic Learning, in: Proceedings of the Thirty-Fifth International Conference on Computer Aided Verification (CAV-2023), volume 13964 of *Lecture Notes in Computer Science*, Springer, Cham, 2023, pp. 390–414.

[80] L. de Alfaro, T. A. Henzinger, R. Majumdar, Symbolic Algorithms for Infinite-State Games, in: Proceedings of the Twelfth International Conference on Computer Aided Verification (CAV-2001), volume 2102 of *Lecture Notes in Computer Science*, Springer, 2001, pp. 536–550.

[81] D. Neider, U. Topcu, An Automaton Learning Approach to Solving Safety Games over Infinite Graphs, in: Proceedings of the Twenty-second International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS-2016), volume 9636 of *Lecture Notes in Computer Science*, Springer, 2016, pp. 204–221.

[82] T. Cachat, Symbolic Strategy Synthesis for Games on Pushdown Graphs,

in: Proceedings of the Twenty-ninth International Colloquium on Automata, Languages and Programming (ICALP-2002), volume 2380 of *Lecture Notes in Computer Science*, Springer, 2002, pp. 704–715.

[83] K. Chatterjee, N. Fijalkow, Infinite-state games with finitary conditions, in: Proceedings of the Thirty-First EACSL Annual Conference on Computer Science Logic (CSL-2013), Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2013, pp. 181–196.

[84] T. Cachat, Uniform Solution of Parity Games on Prefix-Recognizable Graphs, Electronic Notes in Theoretical Computer Science 68 (2003) 71–84.

[85] T. Beyene, S. Chaudhuri, C. Popeea, A. Rybalchenko, A Constraint-based Approach to Solving Games on Infinite Graphs, in: Proceedings of the Forty-First ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL-2014), 2014, pp. 221–233.

[86] A. Farzan, Z. Kincaid, Strategy Synthesis for Linear Arithmetic Games, Proceedings of the ACM on Programming Languages 2 (2017) 1–30.

[87] C. Baier, N. Coenen, B. Finkbeiner, F. Funke, S. Jantsch, J. Siber, Causality-based game solving, in: Proceedings of the Thirty-Third International Conference on Computer Aided Verification (CAV-2021), volume 12759 of *Lecture Notes in Computer Science*, Springer, 2021, pp. 894–917.

[88] M. Faella, G. Parlato, Reachability Games Modulo Theories with a Bounded Safety Player, in: Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI-2023), 2023, pp. 6330–6337.

[89] T. A. Beyene, C. Popeea, A. Rybalchenko, Solving Existentially Quantified Horn Clauses, in: Proceedings of the Twenty-Fifth International Conference on Computer Aided Verification (CAV-2013), volume 8044 of *Lecture Notes in Computer Science*, Springer, 2013, pp. 869–882.

[90] A. Farzan, Z. Kincaid, Linear Arithmetic Satisfiability via Strategy Improvement, in: Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 735–743.

[91] N. Daniel, M. Oliver, Learning-based Synthesis of Safety Controllers, in: Proceedings of the Nineteenth International Conference on Formal Methods in Computer Aided Design (FMCAD-2019), IEEE, 2019, pp. 120–128.

[92] S. Samuel, D. D'Souza, R. Komondoor, Symbolic Fixpoint Algorithms for Logical LTL Games, in: Proceedings of the Thirty-Eighth IEEE/ACM International Conference on Automated Software Engineering (ASE-2023), IEEE, 2023, pp. 698–709.

[93] P. Heim, R. Dimitrova, Solving Infinite-state Games via Acceleration, Proceedings of the ACM on Programming Languages 8 (2024) 1696–1726.

[94] A.-K. Schmuck, P. Heim, R. Dimitrova, S. P. Nayak, Localized Attractor Computations for Infinite-State Games, in: Proceedings of the Thirty-Sixth International Conference on Computer Aided Verification (CAV-2024),, volume 14683 of *Lecture Notes in Computer Science*, Springer, 2024, pp. 135–158.

[95] S. Azzopardi, N. Piterman, L. Di Stefano, G. Schneider, Full ltl synthesis over infinite-state arenas, in: Proceedings of the Thirty-Seventh International Conference on Computer Aided Verification (CAV-2025), Springer, 2025, pp. 274–297.

[96] H. J. Levesque, Planning with Loops, in: Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI-2005), 2005, pp. 509–515.

[97] Y. Hu, H. J. Levesque, A Correctness Result for Reasoning about One-Dimensional Planning Problems, in: Proceedings of the Twelfth Inter-

national Conference on Principles of Knowledge Representation and Reasoning (KR-2010), 2010, pp. 362–371.

[98] Y. Hu, G. D. Giacomo, Generalized Planning: Synthesizing Plans that Work for Multiple Environments, in: Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI-2011), 2011, pp. 918–923.

[99] S. Srivastava, N. Immerman, S. Zilberstein, A new representation and associated algorithms for generalized planning, Artificial Intelligence 175 (2011) 615–647.

[100] S. Srivastava, N. Immerman, S. Zilberstein, Applicability conditions for plans with loops: Computability results and algorithms, Artificial Intelligence 191 (2012) 1–19.

[101] B. Bonet, H. Palacios, H. Geffner, Automatic Derivation of Memoryless Policies and Finite-State Controllers Using Classical Planners, in: Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS-2009), 2009, pp. 34–41.

[102] B. Bonet, H. Geffner, Policies that Generalize: Solving Many Planning Problems with the Same Policy, in: Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-2015), 2015, pp. 2798–2804.

[103] J. Segovia, S. Jiménez, A. Jonsson, Generalized planning with procedural domain control knowledge, in: Proceedings of the International Conference on Automated Planning and Scheduling, 2016, pp. 285–293.

[104] J. Segovia-Aguas, S. Jiménez, A. Jonsson, Computing programs for generalized planning using a classical planner, Artificial Intelligence 272 (2019) 52–85.

[105] J. Segovia-Aguas, S. J. Celorrio, A. Jonsson, Generalized Planning as Heuristic Search, in: Proceedings of the Thirty-First International Con-

ference on Automated Planning and Scheduling (ICAPS-2021), 2021, pp. 569–577.

[106] S. Srivastava, S. Zilberstein, N. Immerman, H. Geffner, Qualitative Numeric Planning, in: Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence (AAAI-2011), 2011, pp. 1010–1016.

[107] B. Bonet, G. Francès, H. Geffner, Learning Features and Abstract Actions for Computing Generalized Plans, in: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-2019), 2019, pp. 2703–2710.

[108] B. Bonet, R. Fuentetaja, Y. E-Martín, D. Borrajo, Guarantees for Sound Abstractions for Generalized Planning, in: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI-2019), 2019, pp. 1566–1573.

[109] A. Lomuscio, J. Michaliszyn, An Abstraction Technique for the Verification of Multi-Agent Systems Against ATL Specifications, in: Proceedings of the Fourteenth International Conference on Principles of Knowledge Representation and Reasoning (KR-2014), 2014, pp. 428–437.

[110] A. Lomuscio, J. Michliszyn, Verification of multi-agent systems via predicate abstraction against atlk specifications, in: Proceedings of the Eighteen International Conference on Autonomous Agents and Multiagent Systems (AAMAS-2016), 2016, pp. 662–670.

[111] F. Belardinelli, A. Lomuscio, J. Michaliszyn, Agent-based refinement for predicate abstraction of multi-agent systems, in: Proceedings of the Twenty-Second European Conference on Artificial Intelligence (ECAI-2016), 2016, pp. 286–294.

[112] F. Belardinelli, A. Ferrando, V. Malvone, An abstraction-refinement framework for verifying strategic properties in multi-agent systems with imperfect information, Artificial Intelligence 316 (2023) 103847.

[113] G. De Giacomo, A. Murano, S. Rubin, A. D. Stasio, Imperfect-Information Games and Generalized Planning, in: Proceedings of the Twenty-fifth International Joint Conference on Artificial Intelligence (IJCAI-2016), 2016, pp. 1037–1043.